



Université de Tunis El Manar – Faculté des Sciences de Tunis

Département Informatique

En partenariat avec **Tunisie Telecom**

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de
Licence Nationale en Informatique

Sécurisation des flux de communication pour un écosystème IoT simulé

Architecture E2E – PKI – TLS/mTLS – SIEM – ML

Réalisé par :

Amine GHANNOUCHI

Encadrante universitaire : Mme. **ELLOUZE NOURHENE** – FST

Encadrant entreprise : M. Moez **KHLIFI** – Tunisie Telecom

Année universitaire **2025 – 2026**

Dédicace

*À mes parents, pour leur soutien indéfectible
et leur confiance tout au long de ce parcours.*

*À mes enseignants de la Faculté des Sciences de Tunis
qui m'ont transmis la passion de l'informatique.*

*À tous ceux qui œuvrent pour la sécurité
des systèmes d'information de demain.*

Remerciements

Ce travail n'aurait pu aboutir sans le concours de nombreuses personnes à qui je tiens à exprimer ma gratitude.

Je remercie tout d'abord **Mme. ELLOUZE NOURHENE**, encadrante universitaire à la Faculté des Sciences de Tunis, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de ce projet.

Mes remerciements vont également à **M. Moez KHLIFI** de *Tunisie Telecom* pour m'avoir accueilli au sein de son équipe, orienté vers des problématiques réelles de l'industrie télécom et soutenu dans mes expérimentations techniques.

Je remercie l'ensemble du corps enseignant du **Département Informatique de la FST** pour la qualité de la formation dispensée.

Un grand merci aux équipes des projets open-source **Wazuh**, **HashiCorp Vault**, **Mosquitto** et **Suricata** dont les documentations exhaustives ont grandement facilité l'implémentation.

Enfin, je remercie ma famille et mes amis pour leur encouragement constant durant cette année académique.

Table des matières

Dédicace	i
Remerciements	iii
Liste des acronymes	xii
Introduction générale	1
1 Contexte général du projet	3
1.1 Introduction	3
1.2 Cadre général du projet	3
1.3 L'IoT dans les réseaux de télécommunications	3
1.3.1 Croissance et enjeux du marché IoT	3
1.3.2 Rôle des opérateurs télécom dans l'écosystème IoT	4
1.3.3 Approches de sécurisation disponibles pour les opérateurs	5
1.4 Présentation de l'organisme d'accueil	5
1.4.1 Service d'accueil	6
1.4.2 Positionnement de Tunisie Telecom face aux enjeux IoT	7
1.5 Analyse de l'existant	7
1.5.1 État des lieux de la sécurité IoT dans les déploiements opérateur	7
1.5.2 Diagnostic synthétique	8
1.6 Problématique	9
1.7 Objectifs spécifiques	10
1.8 Périmètre et contraintes du projet	10
1.8.1 Périmètre fonctionnel	10
1.8.2 Contraintes du projet	10
1.8.3 Hypothèses de travail	11
1.9 Démarche méthodologique	12
1.10 Solution proposée (vue d'ensemble)	13
1.11 Conclusion	15
2 État de l'art	16
2.1 Introduction	16
2.2 Menaces et vulnérabilités spécifiques à l'IoT	16
2.2.1 Surface d'attaque étendue	16
2.2.2 OWASP IoT Top 10 et menaces réseau	16
2.2.3 Études de cas de référence	17

2.3	Protocoles applicatifs IoT	17
2.4	Sécurisation du transport : TLS et mTLS	18
2.4.1	TLS 1.3	18
2.4.2	Authentification mutuelle	18
2.5	Gestion des identités : PKI et automatisation	19
2.6	Détection d'intrusion et supervision	19
2.6.1	IDS réseau orienté MQTT	19
2.6.2	SIEM Wazuh	20
2.7	Apprentissage automatique pour la détection d'anomalies	21
2.8	Analyse critique de l'existant	22
2.9	Synthèse du chapitre	22
2.10	Conclusion	22
3	Conception	23
3.1	Introduction	23
3.2	Spécification des besoins	23
3.2.1	Besoins fonctionnels (BF)	23
3.2.2	Besoins non fonctionnels (BNF)	23
3.2.3	Diagramme de cas d'utilisation	24
3.3	Planification des sprints	26
3.4	Product Backlog	27
3.5	Conception UML	28
3.5.1	Diagramme de classes du simulateur	28
3.5.2	Diagramme de séquence : émission d'un certificat client	29
3.5.3	Diagramme d'activité : cycle de publication mTLS	30
3.6	Diagramme de Gantt	32
3.7	Conclusion	32
4	Réalisation	33
4.1	Introduction	33
4.2	Environnement de développement	33
4.3	Architecture globale et topologie réseau	34
4.3.1	Architecture de déploiement	34
4.3.2	Topologie réseau	35
4.4	Infrastructure à clés publiques (Vault)	38
4.4.1	Hierarchie	38
4.4.2	Rôles Vault et émission	40
4.5	Broker MQTT en TLS 1.3 + mTLS	40
4.6	Scénarios d'attaque et contre-mesures	42
4.7	Pipeline d'apprentissage automatique	45

4.7.1	Données et features	45
4.7.2	Modèles	47
4.8	Captures d'écran des interfaces	48
4.9	Conclusion	49
5	Tests, résultats et validation	50
5.1	Introduction	50
5.2	Plan de tests	50
5.3	Résultats de sécurité	50
5.4	Résultats du module ML	51
5.4.1	Isolation Forest	51
5.4.2	Random Forest	52
5.5	Résultats de performance	55
5.5.1	Handshake TLS et latence	55
5.5.2	Débit et RTT	55
5.6	Limites identifiées	57
5.7	Perspectives	57
5.8	Conclusion	58
	Conclusion générale	59
	Références bibliographiques	61
A	Scripts et Code Source	63
A.1	Bootstrap de la PKI — Vault API	63
A.2	Émission du certificat Mosquitto	66
A.3	Génération des certificats devices	66
A.4	Génération du fichier ACL Mosquitto	68
A.5	Tests de connectivité réseau	68
A.6	Tests de performance MQTT	70
A.7	Simulateur de flotte IoT	71
A.8	Analyse des événements Suricata	72
B	Fichiers de Configuration	74
B.1	Configuration MikroTik CHR	74
B.2	Configuration Mosquitto (TLS/mTLS)	76
B.3	Configuration Suricata (MQTT natif)	77
B.4	Règles Suricata personnalisées	78
B.5	Règles Wazuh personnalisées	79
B.6	Dockerfile du simulateur de flotte	81
B.7	Dockerfile Toolbox	81

B.8	Commandes de création des réseaux Docker	82
B.9	Variables d’environnement du projet	83

Table des figures

1.1	Logo de Tunisie Telecom	5
1.2	Organigramme de Tunisie Telecom	6
1.3	Vue d'ensemble de l'architecture de sécurisation proposée. <i>Abréviations : GW = passerelle applicative, FW = pare-feu.</i>	14
3.1	Diagramme de cas d'utilisation de la plateforme de sécurisation IoT . .	25
3.2	Enchaînement des six sprints et leurs dépendances	26
3.3	Diagramme de classes du simulateur de flotte	28
3.4	Séquence d'émission d'un certificat client	29
3.5	Activité d'un dispositif : chargement des secrets, handshake mTLS, pu- blication	31
3.6	Diagramme de Gantt du projet	32
4.1	Architecture de déploiement : stacks Docker, ports et dépendances ef- fectives	34
4.2	Topologie réseau détaillée : VLANs, plan d'adressage IP et politique de filtrage	35
4.3	Flux E2E — Phase 1 : provisionnement de l'identité	36
4.4	Flux E2E — Phase 2 : établissement de la connexion TLS 1.3 mTLS .	36
4.5	Flux E2E — Phase 3 : publication des données IoT	37
4.6	Flux E2E — Phase 4 : détection et supervision	37
4.7	Flux E2E — Phase 5 : rotation automatique des certificats	38
4.8	Hierarchie de la PKI à deux niveaux (CA racine <i>offline</i> + CA intermé- diaire Vault)	39
4.9	<i>Handshake</i> TLS 1.3 avec authentification mutuelle (mTLS)	41
4.10	Chaîne de détection Suricata → Wazuh (IDS+SIEM)	44
4.11	Pipeline d'apprentissage automatique (collecte, features, entraînement, évaluation, SIEM)	46
4.12	Analyse exploratoire — distribution des classes	46
4.13	Analyse exploratoire — matrice de corrélation des features numériques	47
4.14	Vault UI : vue de la hiérarchie PKI (CA racine et CA intermédiaire actives)	48
4.15	Wazuh Dashboard : alertes corrélées issues des scénarios d'attaque . . .	48
4.16	GNS3 : topologie déployée du laboratoire (VMs et conteneurs Docker visibles)	49

5.1	Isolation Forest – matrice de confusion	51
5.2	Isolation Forest – distribution des scores d’anomalie	52
5.3	Random Forest : matrice de confusion et courbe ROC	52
5.4	Random Forest – confusion multi-classes (8 classes)	53
5.5	Random Forest – importance des features	53
5.6	Comparaison synthétique des modèles	54
5.7	Distribution du handshake mTLS	55
5.8	Latence de connexion	55
5.9	Débit applicatif	56
5.10	RTT par taille de message	56
5.11	Comparaison MQTT clair vs MQTT/TLS vs MQTT/mTLS	57

Liste des tableaux

1.1	Comparaison des approches de sécurisation IoT pour les opérateurs . .	5
1.2	Diagnostic des lacunes et briques de réponse correspondantes	9
1.3	Objectifs spécifiques du projet	10
1.4	Périmètre du projet – inclus et exclus	11
1.5	Comparaison Scrum, Kanban et XP	12
1.6	Correspondance lacunes – briques de sécurité – objectifs spécifiques . .	14
2.1	Comparaison des principaux protocoles applicatifs IoT	17
2.2	Comparaison des solutions IDS open source	20
2.3	Comparaison des solutions SIEM	21
3.1	Besoins fonctionnels	23
3.2	Besoins non fonctionnels	24
3.3	Planification des sprints	26
3.4	Product Backlog (extrait priorisé)	27
4.1	Composants de l’environnement de développement	33
5.1	Plan de tests	50
5.2	Taux de détection des scénarios d’attaque (10 rejeux par scénario) . . .	51
5.3	Performances ML (validation 5-fold)	54
B.1	Variables d’environnement principales du projet	83

Liste des acronymes

IoT	Internet of Things – Internet des Objets
TLS	Transport Layer Security
mTLS	Mutual TLS – TLS mutuel (authentification bidirectionnelle)
DTLS	Datagram TLS
PKI	Public Key Infrastructure – Infrastructure à Clés Publiques
CA	Certificate Authority – Autorité de Certification
CSR	Certificate Signing Request
CRL	Certificate Revocation List
OCSP	Online Certificate Status Protocol
MQTT	Message Queuing Telemetry Transport
CoAP	Constrained Application Protocol
HTTP	HyperText Transfer Protocol
AMQP	Advanced Message Queuing Protocol
SIEM	Security Information and Event Management
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
DPI	Deep Packet Inspection
DoS	Denial of Service
DDoS	Distributed Denial of Service
MiTM	Man-in-the-Middle
ML	Machine Learning – Apprentissage Automatique
RF	Random Forest
IF	Isolation Forest

ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
RTT	Round-Trip Time
QoS	Quality of Service
DMZ	DeMilitarized Zone
VLAN	Virtual Local Area Network
GNS3	Graphical Network Simulator 3
VM	Virtual Machine
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ACL	Access Control List
CN	Common Name
SAN	Subject Alternative Name
E2E	End-to-End
RSA	Rivest–Shamir–Adleman
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
OVA	Open Virtual Appliance
TT	Tunisie Telecom
FST	Faculté des Sciences de Tunis
OWASP	Open Worldwide Application Security Project
ENISA	European Union Agency for Cybersecurity
NIST	National Institute of Standards and Technology
RFC	Request For Comments
ADSL	Asymmetric Digital Subscriber Line

LPWAN Low-Power Wide-Area Network

SOC Security Operations Center

ETL Extract Transform Load

BF-1 Besoin Fonctionnel n°1

BF-2 Besoin Fonctionnel n°2

BF-3 Besoin Fonctionnel n°3

BF-4 Besoin Fonctionnel n°4

BF-5 Besoin Fonctionnel n°5

BF-6 Besoin Fonctionnel n°6

BNF-1 Besoin Non Fonctionnel n°1

BNF-2 Besoin Non Fonctionnel n°2

BNF-3 Besoin Non Fonctionnel n°3

BNF-4 Besoin Non Fonctionnel n°4

BNF-5 Besoin Non Fonctionnel n°5

BNF-6 Besoin Non Fonctionnel n°6

O1 Objectif n°1

O2 Objectif n°2

O3 Objectif n°3

O4 Objectif n°4

O5 Objectif n°5

O6 Objectif n°6

O7 Objectif n°7

Introduction générale

L'**Internet des Objets** (IoT, *Internet of Things*) désigne l'interconnexion de dispositifs physiques capables de collecter, transmettre et exploiter des données dans des environnements très variés : villes intelligentes, santé connectée, industrie, agriculture, transport ou infrastructures de télécommunications [1]. Ces dispositifs peuvent être de simples capteurs, des passerelles industrielles, des compteurs intelligents ou des équipements embarqués. Leur rôle est de rapprocher le monde physique des systèmes numériques afin d'automatiser la surveillance, l'analyse et la prise de décision.

Cette évolution apporte cependant de nouveaux risques. Contrairement aux systèmes informatiques classiques, les objets connectés sont souvent nombreux, hétérogènes, parfois peu puissants et rarement administrés manuellement après leur déploiement. Ils communiquent sur des réseaux partagés, publient des données sensibles et s'appuient sur des protocoles applicatifs légers comme le **Message Queuing Telemetry Transport** (MQTT) ou le **Constrained Application Protocol** (CoAP). La moindre faiblesse d'authentification, de chiffrement ou de segmentation peut donc exposer l'ensemble de l'écosystème à l'interception, à l'usurpation d'identité, au déni de service ou à l'exfiltration de données.

La sécurité des communications IoT doit ainsi être pensée de bout en bout. Elle ne se limite pas à chiffrer un canal réseau : elle implique la gestion fiable des identités, l'émission et la rotation de certificats, l'authentification mutuelle entre dispositifs et services, la restriction des accès par politiques, ainsi qu'une supervision capable de détecter les comportements anormaux. Dans un contexte opérateur, ces exigences deviennent encore plus importantes, car l'infrastructure peut héberger plusieurs services, plusieurs clients et des volumes élevés de messages.

Ce projet de fin d'études vise à concevoir, implémenter et valider une architecture complète de sécurisation des flux de communication dans un écosystème Internet des Objets (IoT) simulé, en adoptant une approche de défense en profondeur. Contrairement aux approches classiques souvent limitées à un seul niveau de sécurité (réseau ou applicatif), la solution proposée intègre de manière cohérente plusieurs mécanismes complémentaires : une gestion automatisée des identités via une **Infrastructure à Clés Publiques** (PKI, *Public Key Infrastructure*), un chiffrement systématique des communications basé sur le **Transport Layer Security** (TLS) avec authentification mutuelle (mTLS, *mutual TLS*), un contrôle d'accès fin au niveau applicatif, ainsi qu'une chaîne de supervision combinant détection par signatures (**Système de Détection d'Intrusion**, IDS, couplé à un **Système de Gestion des Informations et Événements de Sécurité**, SIEM) et détection comportementale basée sur l'**apprentissage automatique** (ML, *Machine Learning*). L'apport principal de ce travail réside dans l'intégration bout-en-bout de ces briques hétérogènes au sein d'un environnement re-

productible, permettant non seulement de sécuriser les communications IoT, mais aussi de corréler les événements de sécurité et de détecter des attaques avancées difficilement identifiables par des approches traditionnelles. Cette approche fournit ainsi une architecture expérimentale représentative des contraintes d'un environnement opérateur, tout en mettant en évidence les compromis entre sécurité, performance et scalabilité.

Le rapport est organisé en cinq chapitres. Le chapitre 1 présente le contexte général du projet, l'organisme d'accueil, la problématique, les objectifs et la solution proposée. Le chapitre 2 expose l'état de l'art relatif aux menaces IoT, aux protocoles, aux mécanismes de sécurité et aux outils de supervision. Le chapitre 3 détaille la conception : besoins, choix méthodologique, planification Scrum, backlog et diagrammes en **Unified Modeling Language** (UML). Le chapitre 4 décrit la réalisation technique de l'architecture, les scénarios d'attaque, le pipeline d'**apprentissage automatique** (ML, *Machine Learning*) et les captures d'interfaces. Enfin, le chapitre 5 présente les tests, les résultats, les limites et les perspectives. Une conclusion générale synthétise ensuite les apports du projet.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre présente le cadre général dans lequel s’inscrit le projet de fin d’études. Il situe d’abord le travail dans le contexte du marché IoT et du rôle spécifique des opérateurs télécom, puis expose le cadre de Tunisie Telecom, avant de dresser un diagnostic des lacunes de sécurité qui justifient le projet. La problématique est ensuite formalisée, les objectifs spécifiques définis, le périmètre et les contraintes précisés, et enfin la solution proposée présentée dans ses grandes lignes.

1.2 Cadre général du projet

Dans le cadre de ce travail, l’objectif est de construire un prototype expérimental capable de sécuriser les flux de communication d’un écosystème IoT simulé. Le projet s’appuie sur une architecture représentative d’un environnement opérateur : segmentation réseau, services de confiance, broker MQTT, supervision de sécurité et analyse comportementale. Cette approche permet de valider les mécanismes proposés avant toute projection vers un déploiement réel à plus grande échelle.

1.3 L’IoT dans les réseaux de télécommunications

1.3.1 Croissance et enjeux du marché IoT

L’Internet des Objets constitue l’une des évolutions technologiques les plus structurantes de la décennie actuelle. Selon l’ENISA [2], le nombre de dispositifs connectés actifs dans le monde poursuit une croissance soutenue, portée par la multiplication des cas d’usage dans l’industrie, les villes intelligentes, la santé connectée, les infrastructures énergétiques et les réseaux de transport. Cette expansion génère un volume massif de données dont la collecte, le transport et le traitement reposent en très grande partie sur les infrastructures des opérateurs télécom.

Cette croissance s’accompagne cependant d’une complexification notable des exigences de sécurité. Meneghello et al. [1] soulignent que les dispositifs IoT présentent structurellement des vulnérabilités liées à leur conception : ressources processeur et mémoire limitées rendant difficile l’intégration de mécanismes cryptographiques complets, cycles de mise à jour longs ou inexistants, et exposition à des réseaux partagés peu maîtrisés. Ces fragilités sont exploitées de manière croissante par des acteurs malveillants,

comme l'illustre l'attaque Mirai [3], qui a mobilisé plusieurs centaines de milliers d'objets compromis — équipements de vidéosurveillance, routeurs domestiques, passerelles diverses — pour générer des attaques par déni de service dépassant le téraoctet par seconde et mettre hors ligne des services critiques à l'échelle mondiale.

1.3.2 Rôle des opérateurs télécom dans l'écosystème IoT

Les opérateurs télécom occupent une position centrale et particulière dans l'écosystème IoT : ils fournissent la connectivité (2G/4G/LTE/5G, NB-IoT, LoRaWAN), hébergent les plateformes de gestion des dispositifs et proposent des services à valeur ajoutée incluant la supervision, l'analyse des données et la sécurité managée. Ce positionnement crée cependant des responsabilités spécifiques qui n'ont pas d'équivalent dans les déploiements IoT privés :

- **Multi-tenance et isolation** : un même réseau et une même plateforme IoT peuvent héberger simultanément les objets de plusieurs clients verticaux distincts (énergie, transport, santé, industrie). L'isolation rigoureuse des flux entre locataires est une exigence de sécurité critique que les architectures IoT génériques ne prennent pas toujours en charge nativement ;
- **Volumétrie et passage à l'échelle** : les messages IoT peuvent atteindre des millions de publications par heure sur un seul cluster de brokers. Les mécanismes de sécurité — chiffrement, authentification, contrôle d'accès — doivent être conçus pour fonctionner à cette échelle sans dégrader la qualité de service ni la latence applicative ;
- **Hétérogénéité du parc de terminaux** : l'opérateur doit gérer des dispositifs aux capacités très différentes — microcontrôleurs disposant de quelques kilooctets de RAM, passerelles industrielles sous Linux embarqué, modems cellulaires NB-IoT — en appliquant des politiques de sécurité cohérentes à l'ensemble du parc ;
- **Responsabilité contractuelle et réglementaire** : une fuite ou une compromission de flux IoT clients expose l'opérateur à des risques réglementaires (protection des données personnelles, responsabilité de service) et à des impacts réputationnels majeurs, le contraignant à démontrer un niveau de sécurité maîtrisé et auditable.

Ces contraintes font de la sécurisation des flux IoT un enjeu opérationnel de premier ordre pour les opérateurs télécom, bien au-delà d'une simple exigence technique ponctuelle. Elles motivent directement la conception d'une architecture dédiée, reproductible et validée expérimentalement dans le cadre de ce projet.

1.3.3 Approches de sécurisation disponibles pour les opérateurs

Face à ces enjeux, plusieurs approches de sécurisation sont envisageables. Le tableau 1.1 présente une comparaison structurée des trois grandes familles de solutions.

Tab. 1.1 : Comparaison des approches de sécurisation IoT pour les opérateurs

Approche	Avantages	Limites pour un opérateur
Plateformes cloud commerciales (AWS IoT, Azure IoT Hub)	Déploiement rapide, fonctionnalités riches, mises à jour gérées	Dépendance fournisseur, souveraineté des données, coûts récurrents variables
Distributions open source intégrées (ThingsBoard, Eclipse Kura)	Open source, personnalisable, large communauté	TLS souvent optionnel, PKI non intégrée, supervision et corrélation limitées
Architecture sur mesure (composants open source)	Contrôle total, souveraineté, adaptabilité au contexte opérateur	Complexité initiale de mise en œuvre, nécessite expertise interne

Pour un opérateur télécom souhaitant conserver la maîtrise de son infrastructure, répondre aux exigences réglementaires locales et éviter tout verrouillage fournisseur, une architecture sur mesure fondée sur des composants open source éprouvés constitue l'approche la plus cohérente. C'est ce positionnement qui oriente l'ensemble des choix techniques opérés dans ce projet.

1.4 Présentation de l'organisme d'accueil

Tunisie Telecom est l'opérateur historique de télécommunications en Tunisie. Fondée en 1995, l'entreprise joue un rôle clé dans le développement des infrastructures de communication du pays et dans la fourniture de services de téléphonie, d'Internet et de transmission de données [4]. La figure 1.1 présente le logo de Tunisie Telecom.



Fig. 1.1 : Logo de Tunisie Telecom

L'opérateur propose une large gamme de services destinés aussi bien aux particuliers qu'aux entreprises, incluant :

- la téléphonie fixe et mobile .

- l'accès à Internet haut débit et très haut débit .
- les services de données et de connectivité .
- les solutions cloud et les services numériques.

Grâce à un vaste réseau d'infrastructures couvrant l'ensemble du territoire national, Tunisie Telecom contribue activement à la transformation numérique du pays. L'entreprise investit également dans des technologies émergentes telles que la 5G, l'Internet des objets et les solutions de cybersécurité afin de répondre aux besoins croissants en connectivité et en sécurité.

Dans ce contexte, l'opérateur développe des initiatives visant à renforcer la sécurité des infrastructures réseau et des services numériques, notamment face à l'augmentation des cybermenaces.

1.4.1 Service d'accueil

La structure organisationnelle de Tunisie Telecom est constituée d'une direction générale, de neuf directions centrales et de 24 directions régionales. Chacune des parties prenantes de l'organisation joue un rôle indéniablement important. Cette organisation est illustrée dans la figure 1.2.

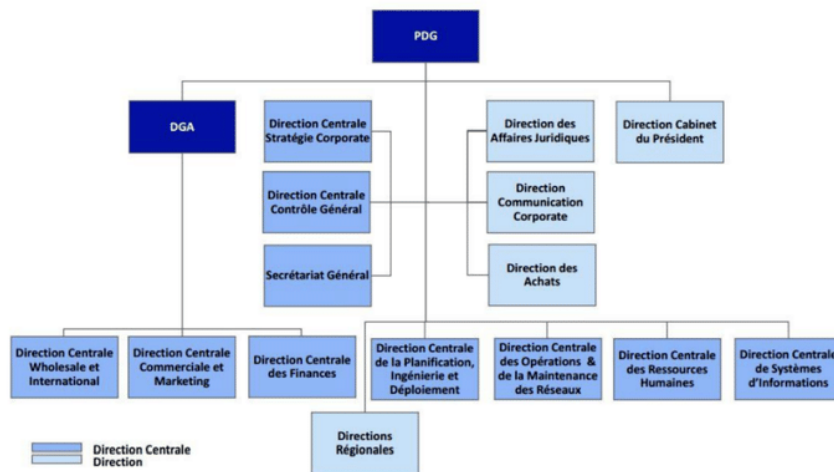


Fig. 1.2 : Organigramme de Tunisie Telecom

Le projet a été réalisé au sein du service technique chargé des infrastructures réseau et de la sécurité des systèmes de communication. Ce service joue un rôle essentiel dans la gestion, la supervision et la protection des infrastructures utilisées pour les services télécom et les plateformes numériques.

Les principales activités du service incluent :

- la gestion et l'administration des infrastructures réseau .

- la supervision du trafic et des performances des systèmes .
- la mise en œuvre de solutions de cybersécurité .
- l'analyse des incidents et des menaces de sécurité .
- l'évaluation et l'intégration de nouvelles technologies.

Dans ce cadre, le service s'intéresse particulièrement aux problématiques de sécurité liées aux architectures modernes telles que les environnements cloud, les réseaux virtualisés et les écosystèmes IoT.

Le stage réalisé dans ce service offre ainsi l'opportunité de travailler sur des problématiques concrètes liées à la sécurisation des communications dans les systèmes IoT.

1.4.2 Positionnement de Tunisie Telecom face aux enjeux IoT

Dans le cadre de sa stratégie de transformation numérique, *Tunisie Telecom* développe progressivement des offres dans le domaine de l'IoT et des services connectés. L'opérateur est confronté aux mêmes impératifs de sécurisation que ses homologues internationaux, dans un contexte supplémentaire de contraintes réglementaires nationales et de ressources techniques à mobiliser de manière optimale.

Le service d'accueil est particulièrement sensibilisé aux risques liés à l'interconnexion de dispositifs IoT sur les infrastructures de l'opérateur : hétérogénéité des équipements connectés, absence de standards unifiés d'authentification à l'échelle du parc, difficultés de supervision des flux applicatifs de type MQTT et manque d'outils permettant la détection de comportements anormaux au niveau applicatif. Ces constats constituent le point de départ direct du projet et justifient la construction d'un démonstrateur expérimental permettant de valider une approche intégrée avant tout passage à l'échelle opérationnel.

1.5 Analyse de l'existant

1.5.1 État des lieux de la sécurité IoT dans les déploiements opérateur

L'analyse conduite en amont de ce projet, combinant l'étude des pratiques observées dans des déploiements IoT typiques et les enseignements publiés par l'ENISA [2] et le NIST [5], met en évidence plusieurs lacunes de sécurité récurrentes dans ce type d'environnement :

- **Transport non systématiquement chiffré** : le protocole MQTT est fréquemment déployé sur le port 1883 (texte clair), sans TLS, par souci de simplicité ou de performance perçue. Un attaquant positionné sur le segment réseau peut alors capturer l'intégralité des messages publiés, y compris les données métier sensibles ;

- **Authentification unidirectionnelle ou absente** : le broker ne vérifie généralement pas le certificat du client. Les dispositifs s'authentifient souvent par un simple couple identifiant/mot de passe, voire sans aucun mécanisme d'authentification, rendant toute usurpation d'identité triviale ;
- **Absence de PKI dédiée** : lorsque des certificats existent, ils sont fréquemment auto-signés, non révocables automatiquement et gérés de façon ad hoc, sans cycle de vie défini ni procédure de révocation d'urgence en cas de compromission ;
- **Contrôle d'accès applicatif insuffisant** : les listes de contrôle d'accès aux topics MQTT sont rarement mises en œuvre, permettant à tout dispositif connecté d'accéder à l'ensemble des canaux de publication et de souscription, indépendamment de son rôle légitime ;
- **Supervision fragmentée** : les outils de monitoring réseau et applicatif ne communiquent pas entre eux, rendant difficile la corrélation d'événements de sécurité. La détection d'une attaque nécessite une analyse manuelle croisée de plusieurs sources de journaux hétérogènes ;
- **Absence de détection comportementale** : les mécanismes de détection d'intrusion existants se limitent aux signatures connues. Ils ne permettent pas de détecter des anomalies comportementales subtiles telles que l'utilisation d'un certificat valide depuis un contexte inhabituel, une dérive de fréquence de publication ou une exfiltration lente via des topics légitimes.

1.5.2 Diagnostic synthétique

Le tableau 1.2 synthétise ce diagnostic en mettant en regard chaque lacune identifiée avec la brique de sécurité retenue dans la solution, établissant ainsi le lien direct entre le contexte opérationnel observé et les choix de conception du projet.

Ce diagnostic structuré constitue la justification directe des choix techniques opérés dans ce projet : chaque composant de la solution répond à une lacune précisément identifiée. Cette correspondance garantit la pertinence de l'architecture proposée au-delà d'une simple accumulation d'outils technologiques.

Tab. 1.2 : Diagnostic des lacunes et briques de réponse correspondantes

Lacune identifiée	Brique de réponse retenue
Flux MQTT en clair, sans chiffrement de bout en bout	TLS 1.3 obligatoire sur Mosquitto (port 8883 uniquement)
Authentification client absente ou basée sur mot de passe faible	mTLS : certificat X.509 client obligatoire, vérifié par le broker
Gestion manuelle et non automatisée des certificats	PKI HashiCorp Vault à deux niveaux avec API d'émission et révocation
Accès applicatifs non restreints par dispositif	ACL Mosquitto liée au <i>Common Name</i> du certificat client
Supervision réseau et applicative déconnectées	IDS Suricata (réseau + MQTT) intégré au SIEM Wazuh (corrélation)
Absence de détection comportementale	Module ML IsolationForest (non supervisé) + Random Forest (classifié)

1.6 Problématique

L'écosystème d'un opérateur télécom présente plusieurs spécificités qui rendent la sécurisation des flux IoT particulièrement délicate :

- **Hétérogénéité** des dispositifs (microcontrôleurs ESP32, passerelles industrielles, modems cellulaires) et des protocoles applicatifs (MQTT, CoAP, AMQP, LwM2M) .
- **Volumétrie** massive et asynchrone des messages (publications périodiques, événements rares mais critiques) .
- **Contraintes embarquées** : ressources CPU/mémoire limitées, batteries, latence radio variable .
- **Multi-tenance** et segmentation réseau : un même opérateur héberge plusieurs clients verticaux qui ne doivent jamais accéder aux flux des autres .
- **Gestion à grande échelle** d'identités cryptographiques (certificats X.509) pour des dizaines de milliers d'objets, avec des cycles de vie variables .

La problématique centrale de ce projet de fin d'études peut être formulée ainsi :

Problématique

Comment concevoir, déployer et valider, dans un environnement simulé représentatif d'un opérateur télécom, une architecture de sécurisation **de bout en bout** des flux de communication d'un écosystème IoT, qui combine : (i) un chiffrement et une authentification mutuelle systématiques entre objets et infrastructures . (ii) une gestion automatisée du cycle de vie des certificats . (iii) une supervision active des menaces

par corrélation d’alertes réseau et apprentissage automatique . (iv) tout en restant compatible avec les contraintes de performance et de scalabilité du domaine ?

1.7 Objectifs spécifiques

À partir de cette problématique, sept objectifs spécifiques (O1–O7) structurent l’ensemble des travaux Qui sont présentés dans le tableau 1.1 :

Tab. 1.3 : Objectifs spécifiques du projet

ID	Objectif
O1	Concevoir une architecture IoT segmentée et reproductible, déployable dans un environnement simulé.
O2	Déployer une infrastructure à clés publiques (PKI) automatisée fondée sur HashiCorp Vault.
O3	Imposer un chiffrement TLS 1.3 avec authentification mutuelle (mTLS) sur le broker MQTT.
O4	Simuler une flotte hétérogène de 50 dispositifs IoT et générer un trafic réaliste mêlant comportements légitimes et attaques.
O5	Mettre en place une analyse réseau native MQTT au moyen d’un IDS Suricata.
O6	Centraliser et corréler les événements de sécurité dans un SIEM Wazuh.
O7	Développer un module de détection d’anomalies par apprentissage automatique (IsolationForest et Random Forest) et évaluer son apport.

Ces objectifs traduisent opérationnellement la problématique et structurent les différentes phases du projet.

1.8 Périmètre et contraintes du projet

1.8.1 Périmètre fonctionnel

Ce projet se concentre sur la sécurisation des communications IoT au niveau du transport et de l’application, depuis la gestion des identités cryptographiques des dispositifs jusqu’à la détection comportementale des anomalies. Le tableau 1.4 précise explicitement ce qui est inclus et exclu du périmètre, afin d’éviter toute ambiguïté dans l’interprétation des résultats présentés.

1.8.2 Contraintes du projet

La réalisation de ce projet s’est inscrite dans un ensemble de contraintes techniques, temporelles et opérationnelles qui ont directement influencé les choix architecturaux :

- **Contrainte matérielle** : l’ensemble de l’infrastructure est simulé sur une station Windows 10/11 dotée de 16 Go de RAM minimum, sans accès à un environnement

Tab. 1.4 : Périmètre du projet – inclus et exclus

Inclus dans le périmètre	Exclu du périmètre
Segmentation réseau par VLANs sous GNS3	Déploiement en environnement de production réel
PKI à deux niveaux avec HashiCorp Vault	Gestion des mises à jour firmware des objets (OTA)
Broker MQTT en TLS 1.3 + mTLS avec ACL par topic	Sécurité de la couche matérielle des dispositifs
Simulation d'une flotte de 50 dispositifs hétérogènes	Flotte supérieure à 50 dispositifs (contrainte ressources hôte)
Détection IDS Suricata + corrélation SIEM Wazuh	Intégration aux outils SOC existants de Tunisie Telecom
Pipeline ML hors ligne (IsolationForest, Random Forest)	Scoring ML en temps réel sur flux de données en streaming
Évaluation des performances (latence, débit mTLS)	Tests de charge en environnement radio réel (4G/5G/NB-IoT)

cloud dédié ni à des équipements physiques IoT réels. Cette contrainte délimite la taille de la flotte simulable et le niveau de réalisme des mesures de performance ;

- **Contrainte temporelle** : le stage d'une durée de six mois impose une priorisation stricte des livrables. La méthodologie Scrum, adoptée pour ce projet et décrite dans la section suivante, garantit que les fonctionnalités critiques (PKI, mTLS, détection) sont livrées et testées en priorité ;
- **Contrainte de reproductibilité** : l'ensemble de l'environnement doit être reconstituable depuis zéro à partir des scripts et configurations versionnés, sans intervention manuelle non documentée ;
- **Contrainte de souveraineté technologique** : tous les composants retenus sont open source, afin de garantir la transparence, la maîtrise du code et l'absence de dépendance envers un fournisseur commercial.

1.8.3 Hypothèses de travail

Trois hypothèses structurent l'expérimentation conduite dans ce projet :

1. Le jeu de données utilisé pour l'entraînement des modèles de ML (76 000 messages répartis en huit classes : une classe normale et sept catégories d'attaques) est représentatif des profils de trafic rencontrés dans un écosystème IoT réel [6] ;
2. Les performances mesurées dans l'environnement simulé (latence du handshake mTLS, débit du broker) constituent des ordres de grandeur valides pour une éva-

luation comparative, sans prétention à reproduire exactement un réseau radio réel avec ses contraintes de propagation et de mobilité ;

3. La détection par règles statiques (Suricata + Wazuh) et la détection par apprentissage automatique sont traitées comme des couches complémentaires et non substituables, chacune couvrant un sous-ensemble différent du spectre des menaces.

1.9 Démarche méthodologique

Le pilotage d'un projet technique de cette envergure dans un délai académique contraint requiert une méthodologie explicitement choisie. Trois cadres agiles ont été évalués : **Scrum** [7], **Kanban** [8] et **Extreme Programming (XP)** [9]. Le tableau 1.5 en présente la comparaison sur sept critères déterminants.

Tab. 1.5 : Comparaison Scrum, Kanban et XP

Critère	Scrum	Kanban	XP
Cadre de travail	Itératif (sprints)	Continu (flux)	Itératif court
Taille équipe	3–9 personnes	Variable	2–12
Gestion des changements	Encadrée (entre sprints)	Très flexible	Flexible
Cycle de livraison	1–4 semaines	Continu	1–3 semaines
Documentation	Modérée (US, backlog)	Faible	Faible
Pratiques d'ingénierie	Indépendantes	Indépendantes	Strictes (TDD, pair-prog)
Courbe d'apprentissage	Modérée	Faible	Élevée
Adaptation au PFE	Très adaptée	Peu adaptée	Peu adaptée

Scrum a été retenu pour trois raisons décisives :

- **Alignement avec les jalons académiques** : les revues de sprint coïncident naturellement avec les points d'avancement imposés (rapport intermédiaire, soutenance), offrant une visibilité régulière sur l'état du projet ;
- **Flexibilité face aux imprévus techniques** : le réajustement du backlog entre deux sprints permet de réagir à un blocage (configuration réseau, problème de certificats) sans remettre en cause l'ensemble du planning ;
- **Maîtrise du périmètre** : la priorisation du backlog garantit que les fonctionnalités essentielles sont livrées en premier, limitant le risque de ne pas finaliser les parties critiques avant la soutenance.

Kanban, dont l’absence de cadence fixe est inadaptée à un projet soumis à des jalons académiques, et XP, dont les pratiques d’ingénierie (TDD, *pair-programming*) sont disproportionnées pour un développement solo, ont été écartés. Six sprints de deux semaines structurent la phase de réalisation ; leur planification détaillée figure au chapitre 3.

1.10 Solution proposée (vue d’ensemble)

Pour répondre à la problématique et atteindre les objectifs ci-dessus, nous proposons une **architecture de défense en profondeur** composée de six briques complémentaires, intégralement déployée dans un laboratoire simulé sous GNS3 [10] et Docker [11] :

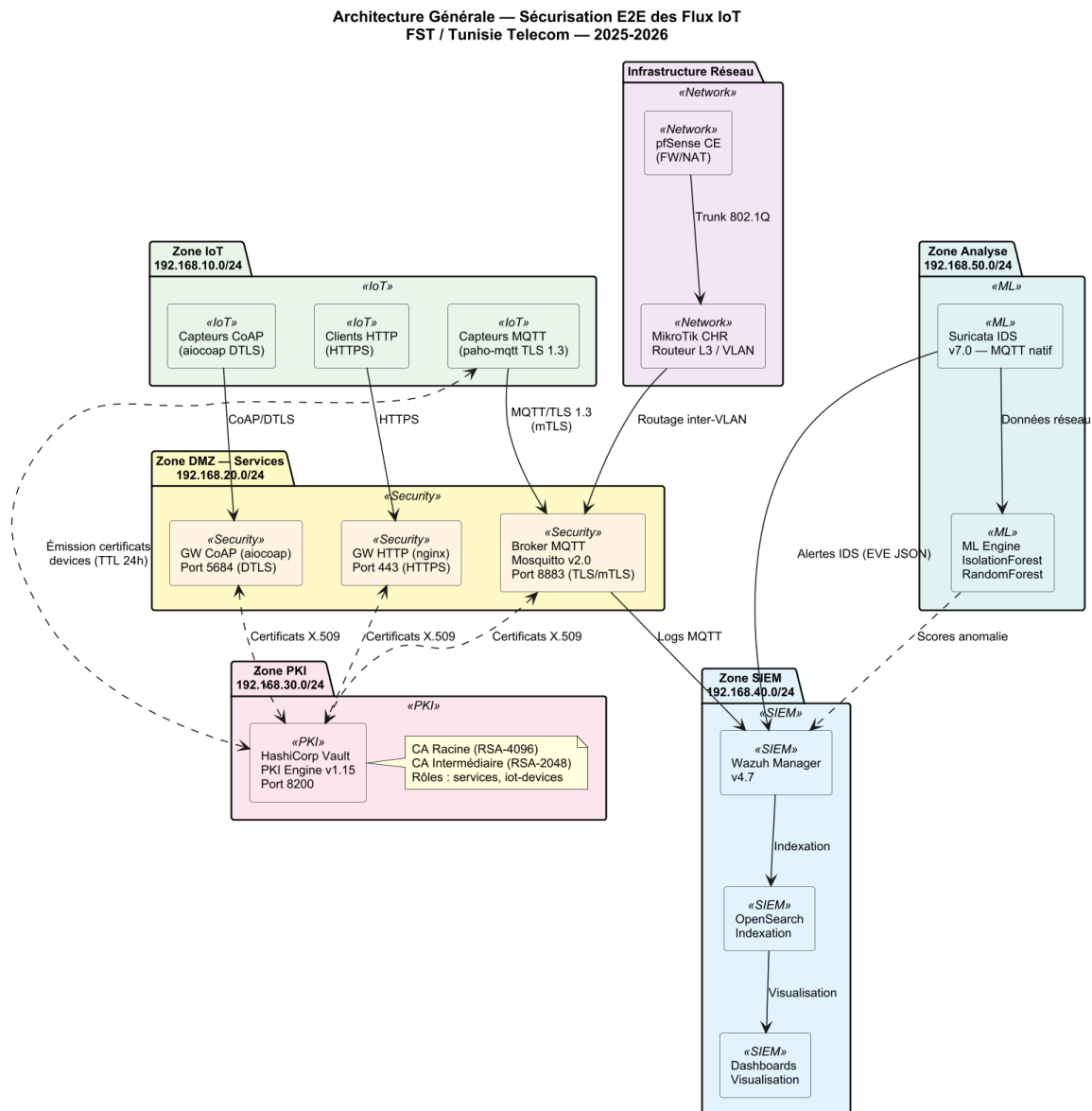
1. Une **infrastructure réseau segmentée** (VLANs IoT, services, monitoring, management) construite autour d’un pare-feu pfSense et d’un routeur MikroTik .
2. Une **PKI automatisée** HashiCorp Vault à deux niveaux (CA racine hors ligne + CA intermédiaire en ligne), capable d’émettre à la demande des certificats X.509 pour les services et les objets [12] .
3. Un **broker MQTT Mosquitto** configuré en TLS 1.3 avec authentification mutuelle obligatoire et listes de contrôle d’accès par topic [13] .
4. Un **simulateur de flotte** de 50 dispositifs Python jouant 76 000 messages issus d’un jeu de données réel, dont sept catégories d’attaques .
5. Une **chaîne de détection** comprenant un IDS Suricata avec règles MQTT personnalisées [14] et un SIEM Wazuh [15] pour la corrélation et la visualisation .
6. Un **pipeline de ML** (IsolationForest [16] et Random Forest [17]) entraîné sur les caractéristiques extraites du trafic, capable de détecter les anomalies non couvertes par les règles statiques.

Le tableau 1.6 établit la correspondance entre les lacunes diagnostiquées à la section 1.5, les briques de la solution et les objectifs spécifiques auxquels elles répondent, assurant ainsi la traçabilité complète entre le diagnostic initial et l’architecture proposée.

La figure 1.3 schématise cette vision globale, qui sera détaillée et justifiée dans les chapitres suivants.

Tab. 1.6 : Correspondance lacunes – briques de sécurité – objectifs spécifiques

Brique de sécurité	Section de réalisation	Obj.
Topologie réseau segmentée (GNS3, pfSense, MikroTik)	§4.3	O1
PKI Vault à deux niveaux (CA racine + CA intermédiaire)	§4.4	O2
Broker Mosquitto TLS 1.3 + mTLS + ACL par topic	§4.5	O3
Simulateur de flotte de 50 dispositifs hétérogènes	§4.6	O4
IDS Suricata avec règles MQTT dédiées	§4.6	O5
SIEM Wazuh (corrélation d'alertes et tableau de bord)	§4.6	O6
Module ML (IsolationForest + Random Forest)	§4.7	O7

Fig. 1.3 : Vue d'ensemble de l'architecture de sécurisation proposée. *Abréviations* : GW = passerelle applicative, FW = pare-feu.

1.11 Conclusion

Ce chapitre a établi le cadre complet du projet en suivant une progression logique : le contexte du marché IoT et le rôle stratégique des opérateurs télécom ont d'abord motivé l'intérêt du sujet ; la présentation de Tunisie Telecom et de son positionnement IoT a ancré le projet dans son contexte industriel ; le diagnostic de l'existant a mis en évidence six lacunes de sécurité précises qui justifient chacune des briques retenues ; la problématique et les sept objectifs spécifiques (O1–O7) ont formalisé les attentes ; le périmètre a délimité les frontières de l'expérimentation ; et la solution proposée a établi la correspondance entre lacunes, briques et objectifs. Le chapitre suivant consolide ce cadrage par l'état de l'art des menaces, protocoles, standards et outils qui fondent scientifiquement les choix architecturaux opérés.

Chapitre 2

État de l’art

2.1 Introduction

Avant de concevoir une architecture de sécurisation des flux IoT, il est indispensable de cadrer le sujet sur trois plans complémentaires : (i) caractériser les **menaces** qui pèsent spécifiquement sur les écosystèmes IoT à grande échelle ; (ii) recenser et évaluer de manière critique les **technologies et protocoles** candidats pour le transport, l’authentification et la supervision ; (iii) analyser les **travaux et solutions existants** afin d’en extraire les bonnes pratiques, d’identifier leurs limites et de positionner notre contribution. Ce chapitre est structuré autour de ces trois axes, suivi d’une analyse critique de l’existant et d’une synthèse positionnant le projet.

2.2 Menaces et vulnérabilités spécifiques à l’IoT

2.2.1 Surface d’attaque étendue

Les écosystèmes IoT présentent une surface d’attaque significativement plus large que les systèmes informatiques classiques. Meneghello *et al.* [1] en distinguent quatre couches : (i) la couche *matérielle* (composants embarqués, JTAG/UART exposés, attaques par canal auxiliaire) . (ii) la couche *firmware* (mises à jour non signées, exécution de code arbitraire) . (iii) la couche *réseau* (transports en clair, protocoles propriétaires faibles) . (iv) la couche *services et écosystème* (API non authentifiées, applications mobiles vulnérables, multi-tenance mal cloisonnée).

2.2.2 OWASP IoT Top 10 et menaces réseau

L’OWASP synthétise dans son *IoT Top 10* [18] les dix risques applicatifs majeurs : mots de passe faibles ou par défaut, services réseau non sécurisés, interfaces écosystème non protégées, mécanisme de mise à jour absent, composants obsolètes, protection insuffisante de la vie privée, transferts et stockage non chiffrés, absence de gestion des dispositifs, paramètres par défaut non sûrs, absence de hardening physique. Dans ce projet, nous nous concentrons sur les menaces **réseau** et **applicatives** : interception (*sniffing*), *man-in-the-middle*, force brute, exfiltration applicative, déni de service, vol/abus de certificats.

2.2.3 Études de cas de référence

L’analyse de l’attaque **Mirai** [3], [19] illustre toutes ces faiblesses simultanément : le botnet exploitait des couples identifiant/mot de passe par défaut sur des objets exposés en SSH/Telnet, recrutait des centaines de milliers de dispositifs en quelques jours, et générait des attaques DDoS dépassant 1 Tbit/s. Le rapport *ENISA Threat Landscape for IoT* [2] confirme que ces vecteurs restent dominants et identifie comme contre-mesures prioritaires : l’authentification forte, le chiffrement systématique, la segmentation réseau et la supervision active. Ces orientations rejoignent les recommandations du NIST [5] pour la cybersécurité des dispositifs IoT.

Cette cartographie des menaces permet d’identifier les vecteurs prioritaires à contrer dans la solution : interception des flux, usurpation d’identité, déni de service et abus de certificats. Le premier levier sur lequel il est possible d’agir est le **protocole de communication applicatif** lui-même : c’est à ce niveau que la surface d’attaque réseau est déterminée et que les protections de base peuvent être imposées.

2.3 Protocoles applicatifs IoT

Le tableau 2.1 compare quatre protocoles applicatifs représentatifs.

Tab. 2.1 : Comparaison des principaux protocoles applicatifs IoT

Critère	MQTT 5	CoAP	AMQP 1.0	HTTP/REST
Modèle	Pub/Sub	Req/Rep	Pub/Sub + queues	Req/Rep
Transport	TCP/TLS	UDP/DTLS	TCP/TLS	TCP/TLS
Empreinte	Très faible	Très faible	Moyenne	Moyenne/élevée
QoS	0/1/2	Confirmable	0/1	Aucun natif
Sécurité native	TLS+ACL	DTLS+OSCORE	SASL+TLS	TLS+JWT
Maturité IoT	Très large	Large	Industrielle	Universelle

Le choix de **MQTT 5** [20] pour ce projet repose sur trois critères : empreinte mémoire et réseau adaptée aux microcontrôleurs . large adoption dans les écosystèmes industriels et les opérateurs IoT . existence de mécanismes natifs d’authentification (TLS+ACL) facilement combinables avec mTLS.

Le protocole applicatif étant fixé, la question du **chiffrement du canal** devient prépondérante : MQTT, en lui-même, ne garantit ni la confidentialité ni l’authenticité des messages échangés. TLS 1.3 avec authentification mutuelle répond précisément à ces deux exigences, comme le détaille la section suivante.

2.4 Sécurisation du transport : TLS et mTLS

2.4.1 TLS 1.3

Le protocole TLS 1.3 [21] apporte par rapport à TLS 1.2 une simplification significative : suppression des suites cryptographiques obsolètes, négociation en **1-RTT** (et 0-RTT optionnel), *forward secrecy* obligatoire, signatures et échanges de clés modernisés (ECDHE, EdDSA, AES-GCM, ChaCha20-Poly1305). Les recommandations opérationnelles publiées dans la RFC 9325 [22] guident le choix des paramètres et la durée de vie des certificats.

2.4.2 Authentification mutuelle

Dans un déploiement classique, seul le serveur s'authentifie auprès du client. Le **mTLS** (*mutual TLS*) impose en outre au client de présenter un certificat X.509 valide signé par une autorité de confiance. Cette double authentification permet : (i) d'éviter qu'un objet illégitime se connecte au broker . (ii) de chiffrer chaque session par bout . (iii) de tracer chaque message via l'identité cryptographique de son émetteur. Cette propriété est essentielle pour la mise en place ultérieure d'ACL par certificat dans Mosquitto [13].

L'authentification mutuelle par certificats suppose cependant l'existence d'une infrastructure capable d'**émettre, renouveler et révoquer** ces certificats à l'échelle d'un parc de dispositifs : c'est précisément le rôle d'une PKI automatisée, dont les principes sont analysés dans la section suivante.

Resultat

Positionnement critique : TLS 1.3 + mTLS Le choix de TLS 1.3 en remplacement de TLS 1.2 répond à trois arguments techniques précis : (i) la suppression des suites vulnérables (RC4, DES, RSA pur sans *forward secrecy*) réduit la surface cryptographique exploitable ; (ii) la négociation en 1-RTT diminue la latence du *handshake*, critique pour les dispositifs contraints ; (iii) la *forward secrecy* obligatoire garantit que la compromission ultérieure d'une clé privée ne met pas en péril les sessions passées. L'ajout du mTLS va au-delà du chiffrement : il substitue à une authentification par mot de passe (facilement rejouable) une preuve cryptographique d'identité non duplicable sans accès à la clé privée du dispositif. Cette double garantie – confidentialité *et* authenticité mutuelle – est la seule combinaison capable de fermer simultanément les vecteurs d'interception et d'usurpation identifiés dans la cartographie des menaces.

2.5 Gestion des identités : PKI et automatisation

Une PKI à deux niveaux (CA racine *offline* + CA intermédiaire *online*) constitue la pratique recommandée [5] : la CA racine ne signe que des CA intermédiaires, et reste hors ligne. Les CA intermédiaires signent les certificats opérationnels (services, dispositifs). En cas de compromission d’une CA intermédiaire, seul le sous-arbre concerné doit être révoqué.

L’automatisation de l’émission est indispensable lorsque le parc dépasse quelques dizaines d’objets. **HashiCorp Vault** [12] fournit un *secrets engine pki* qui expose une API HTTP de demande de certificats avec contrôle fin (rôles, ACL, TTL). C’est la solution retenue dans ce projet.

Resultat

Positionnement critique : Vault vs alternatives PKI Trois alternatives à HashiCorp Vault ont été évaluées : (i) **OpenSSL CLI** : complet techniquement mais entièrement manuel ; toute émission de certificat requiert une intervention humaine, ce qui l’exclut dès que le parc dépasse une dizaine de dispositifs ; (ii) **EJBCA (Keyfactor)** : solution PKI d’entreprise mature, mais sa complexité de déploiement et sa courbe d’apprentissage sont disproportionnées pour un projet académique de six mois ; (iii) **Let’s Encrypt / ACME** : idéal pour les certificats TLS publics, mais inadapté à la PKI interne d’un réseau IoT isolé (absence de validation domaine interne, pas de certificats client mTLS natifs). Vault a été retenu car il est le seul à réunir : API REST d’émission automatisée, gestion fine des rôles et TTL, intégration aux pipelines DevOps, et déploiement entièrement on-premise – critère non négociable dans un contexte opérateur télécom soucieux de souveraineté des données.

Même avec une PKI robuste et un chiffrement bout en bout, il n’est pas possible d’exclure tout incident : un certificat peut être volé, un dispositif légitime peut adopter un comportement déviant, ou une configuration insuffisante peut ouvrir un vecteur non anticipé. La **supervision active** – combinant IDS réseau et SIEM – constitue le filet de sécurité complémentaire à ces mécanismes préventifs.

2.6 Détection d’intrusion et supervision

2.6.1 IDS réseau orienté MQTT

Suricata est un IDS/IPS open source qui dispose, depuis la version 6, d’un *parser* natif MQTT capable d’extraire les champs **CONNECT**, **PUBLISH**, **SUBSCRIBE** [14]. Cette capacité permet de définir des règles métier (par exemple : alerter sur un **CONNECT** sans **ClientID** reconnu, ou sur une rafale de **CONNECT** indiquant un *brute force*) qui ne seraient pas accessibles à un IDS générique.

Le tableau 2.2 compare trois solutions IDS open source représentatives sur les critères déterminants pour ce projet.

Tab. 2.2 : Comparaison des solutions IDS open source

Critère	Suricata 6	Snort 3	Zeek 6
Parser MQTT natif	Oui (v6+)	Non	Partiel (scripts)
IDS/IPS actif	Oui	Oui	Non (analyse seule)
Multi-threading	Oui (natif)	Oui (v3)	Oui
Intégration Wazuh	Native (eve.json)	Adaptateur requis	Scripts Lua
Règles Sigma/Suricata	Native	Conversion requise	Non
Maturité IoT	Élevée	Moyenne	Élevée (analytique)
Verdict	Retenu	Non retenu	Complémentaire

Suricata est retenu pour trois raisons décisives : (i) son parser MQTT natif élimine tout besoin de post-traitement applicatif ; (ii) son format de sortie `eve.json` est nativement consommé par Wazuh, réduisant la complexité d’intégration ; (iii) son moteur multi-thread garantit une analyse sans dégradation de débit sur les segments IoT à forte densité de messages.

2.6.2 SIEM Wazuh

Wazuh [15] est une plateforme de *Security Information and Event Management* qui agrège les journaux d’agents endpoint, de sources réseau (Suricata) et applicatifs (Vault, Mosquitto). Elle s’intègre nativement à OpenSearch pour le stockage et au tableau de bord Wazuh Dashboard pour la visualisation et la corrélation. Sa modularité (règles XML, décodeurs personnalisables) en fait un candidat naturel pour la consolidation des alertes IoT.

Le tableau 2.3 compare Wazuh aux deux autres plates-formes SIEM courantes sur les critères déterminants pour un projet IoT académique à contraintes de coût et de souveraineté.

Wazuh est retenu pour sa triple intégration native : avec Suricata (format `eve.json` sans transformation), avec les agents endpoint sur les conteneurs Docker du laboratoire, et avec OpenSearch pour un stockage illimité. La licence open source garantit en outre la souveraineté des données, critère non négociable dans le contexte opérateur télécom.

Les règles IDS, aussi précises soient-elles, ne peuvent couvrir que les attaques **déjà connues et formalisées**. Pour détecter des comportements nouveaux ou subtils – comme l’utilisation d’un certificat valide depuis un contexte comportemental anormal – une couche d’apprentissage automatique est nécessaire. La section suivante en présente les approches retenues.

Tab. 2.3 : Comparaison des solutions SIEM

Critère	Wazuh 4.7	ELK Stack	Splunk Free
Licence	Open source (OSS)	Open source	Propriétaire (limité)
Intégration Suricata	Native (eve.json)	Via Filebeat	Via add-on HEC
Corrélation native	Oui (règles XML)	Non (Elastic SIEM en payant)	Oui (SPL)
Agents endpoint	Oui (Wazuh agent)	Non natif	Oui (UF)
Volume journaux (gratuit)	Illimité	Illimité	500 MB/j
Courbe d’apprentissage	Modérée	Élevée (tuning)	Modérée
Souveraineté données	Totale (on-premise)	Totale	Partielle (cloud)
Verdict	Retenu	Non retenu	Non retenu

2.7 Apprentissage automatique pour la détection d’anomalies

Les approches par règles statiques sont efficaces pour les attaques connues mais peinent à détecter les comportements nouveaux. Hasan *et al.* [6] ont montré qu’un classifieur supervisé entraîné sur des features réseau et applicatives obtient des taux de détection supérieurs à 99 % sur sept catégories d’attaques IoT. Les algorithmes les plus utilisés sont :

- **Isolation Forest** [16] : méthode non supervisée d’isolement, particulièrement efficace en haute dimension et adaptée aux scénarios où les anomalies sont rares .
- **Random Forest** [17] : ensemble d’arbres de décision, robuste, interprétable (*feature importance*) et performant en classification multi-classes.

La mobilisation conjointe de ces deux algorithmes n’est pas arbitraire : elle répond à deux exigences analytiquement distinctes. Isolation Forest est **non supervisé** : il modélise la normalité sans nécessiter de données étiquetées, ce qui le rend indispensable pour détecter les anomalies *inconnues* – typiquement, l’utilisation d’un certificat valide depuis un contexte comportemental inhabituel (scénario S6). Random Forest est **supervisé** : entraîné sur des exemples d’attaques étiquetées, il fournit une classification précise en sept catégories et une mesure d’importance par feature permettant d’identifier les attributs les plus discriminants. Ces deux algorithmes sont donc complémentaires : l’un couvre l’inconnu, l’autre maximise la précision sur le connu. La bibliothèque *scikit-learn* [23] fournit des implémentations matures de ces deux algorithmes . elle constitue le socle de notre pipeline ML.

2.8 Analyse critique de l'existant

Plusieurs approches ont été proposées pour sécuriser les flux IoT : gateways IoT propriétaires (AWS IoT Core, Azure IoT Hub), distributions intégrées (ThingsBoard, Eclipse Kura), ou architectures sur mesure. Notre analyse fait ressortir quatre limitations récurrentes :

1. **Verrouillage fournisseur** pour les solutions cloud commerciales, peu compatible avec une stratégie souveraine d'opérateur télécom .
2. **Couverture partielle** de la sécurité chez les distributions intégrées (souvent TLS oui, mais pas mTLS systématique ni PKI automatisée) .
3. **Faible intégration** entre la couche réseau (IDS) et la couche applicative (broker, ML) .
4. **Reproductibilité limitée** des architectures publiées dans la littérature, rarement accompagnées d'un environnement de simulation complet.

2.9 Synthèse du chapitre

Au regard de cet état de l'art, le présent projet se positionne comme une **architecture de référence open source** pour la sécurisation des flux IoT chez un opérateur télécom, avec quatre contributions distinctives : (1) déploiement de bout en bout reproductible sous GNS3+Docker . (2) PKI Vault entièrement automatisée avec mTLS systématique . (3) chaîne de détection unifiée Suricata + Wazuh avec règles MQTT spécifiques . (4) couplage de cette détection signature avec un module ML (Isolation-Forest + RandomForest) pour couvrir les anomalies non connues.

2.10 Conclusion

Ce chapitre a établi le socle conceptuel du projet en suivant un fil directeur : des menaces spécifiques à l'IoT découlent les exigences de protocole . du choix de MQTT découle le besoin de TLS 1.3 . de TLS avec mTLS découle la nécessité d'une PKI automatisée . de la PKI découle le besoin de supervision IDS/SIEM . des limites des règles statiques découle l'apport de l'apprentissage automatique. Cette logique de causalité guidera la conception présentée au chapitre suivant, où chaque brique technique trouvera sa justification dans l'analyse conduite ici.

Chapitre 3

Conception

3.1 Introduction

Ce chapitre traduit les besoins exprimés au chapitre précédent en une conception structurée. Nous formalisons d’abord les besoins fonctionnels et non fonctionnels, illustrés par le diagramme de cas d’utilisation qui synthétise les interactions entre acteurs et système. La planification en six sprints Scrum — dont la justification méthodologique figure au chapitre 1 — précède la présentation du *Product Backlog*. Enfin, les diagrammes UML de conception et le diagramme de Gantt constituent la référence formelle pour la phase de réalisation.

3.2 Spécification des besoins

3.2.1 Besoins fonctionnels (BF)

Tab. 3.1 : Besoins fonctionnels

ID	Description
BF-1	Émettre, renouveler et révoquer automatiquement des certificats X.509 pour services et dispositifs.
BF-2	Imposer le chiffrement TLS 1.3 et l’authentification mutuelle (mTLS) sur le broker MQTT.
BF-3	Restreindre l’accès aux topics MQTT par dispositif via des listes de contrôle d’accès.
BF-4	Simuler une flotte hétérogène de 50 dispositifs IoT générant un trafic réaliste.
BF-5	Détecter les attaques réseau et applicatives par règles IDS et corrélérer les alertes dans un SIEM.
BF-6	Détecter les anomalies non couvertes par les règles via un modèle d’apprentissage automatique.

3.2.2 Besoins non fonctionnels (BNF)

Ces besoins définissent le *quoi* de la solution : les fonctionnalités attendues et les contraintes opérationnelles à respecter. Le diagramme de cas d’utilisation (figure 3.1) synthétise ces besoins sous forme d’interactions entre les trois acteurs de la plateforme.

Tab. 3.2 : Besoins non fonctionnels

ID	Description
BNF-1	Sécurité : aucun flux IoT en clair . rotation des secrets . séparation CA racine / intermédiaire.
BNF-2	Performance : latence handshake mTLS < 50 ms . débit broker > 500 msg/s par client.
BNF-3	Reproductibilité : l’environnement complet doit être réinstallable à partir des scripts fournis.
BNF-4	Scalabilité : l’architecture doit accepter sans refonte au moins 200 dispositifs.
BNF-5	Maintenabilité : configurations versionnées, modules indépendants, journalisation centralisée.
BNF-6	Observabilité : tableau de bord Wazuh accessible aux analystes en moins de 3 clics.

3.2.3 Diagramme de cas d’utilisation

Trois acteurs principaux interagissent avec la plateforme : le **dispositif IoT** (publie des télémessures via MQTT/mTLS), l’**administrateur** (déploie, configure et maintient l’infrastructure) et l’**analyste sécurité** (consulte les alertes et investigate les incidents). Les flèches d’inclusion (<<include>>) indiquent des sous-cas obligatoires : par exemple, tout acte de publication MQTT *inclut* nécessairement l’étape de négociation mTLS ; toute connexion d’un dispositif *inclut* la validation de son certificat X.509 par la PKI Vault. Les flèches d’extension (<<extend>>) représentent les comportements optionnels : l’IDS Suricata peut *étendre* la réponse en bloquant activement un flux détecté comme malveillant, et le module ML peut *étendre* la détection en signalant une anomalie non couverte par les règles statiques.

La démarche Scrum retenue au chapitre 1 structure la réalisation en six sprints ; la section suivante en détaille la planification et les livrables.

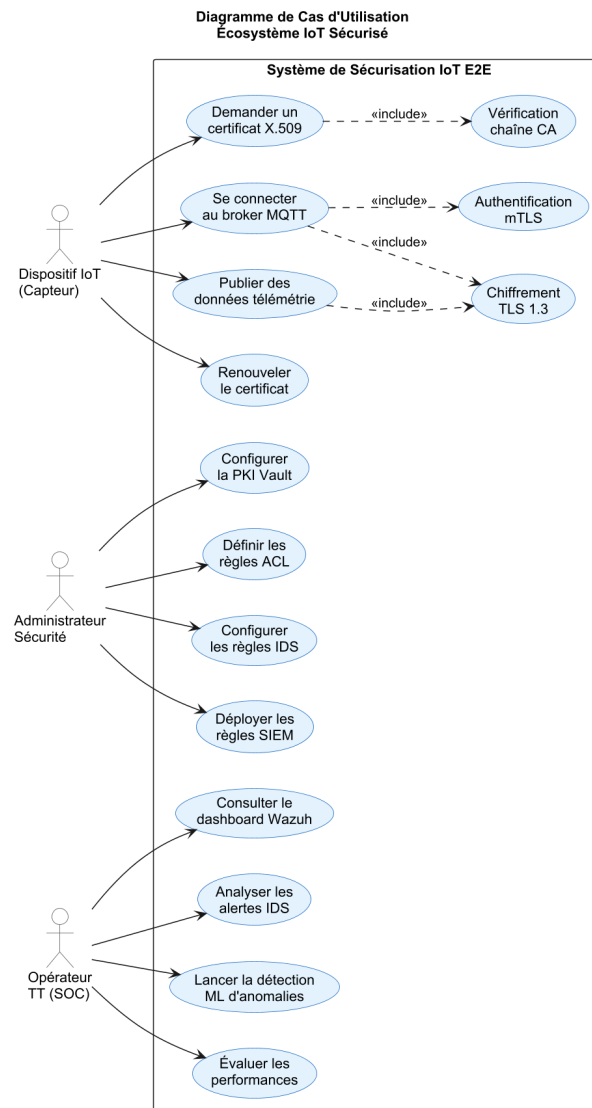


Fig. 3.1 : Diagramme de cas d'utilisation de la plateforme de sécurisation IoT

3.3 Planification des sprints

Six sprints de deux semaines structurent la phase de réalisation. Le tableau 3.3 en présente les objectifs et les livrables. Cette planification précède intentionnellement le *Product Backlog* : elle fournit la grille de lecture qui permet ensuite d'affecter les *User Stories* aux sprints concernés.

Tab. 3.3 : Planification des sprints

Sprint	Durée	Objectif principal
S1 – Cadrage & réseau	2 semaines	Topologie GNS3 segmentée, conteneurs de base, plan d'adressage.
S2 – PKI Vault	2 semaines	Vault déployé, CA racine + intermédiaire, rôles PKI, scripts d'émission.
S3 – Broker mTLS	2 semaines	Mosquitto en TLS 1.3+mTLS, ACL par certificat, tests d'authentification.
S4 – Flotte simulée	2 semaines	50 clients Python, génération de trafic, rejeu de 76 000 messages.
S5 – Détection IDS+SIEM	2 semaines	Règles Suricata MQTT, intégration Wazuh, dashboards.
S6 – ML & bilan	2 semaines	Modèles IsolationForest+RandomForest, évaluation, rédaction finale.

La figure 3.2 illustre l'enchaînement des six sprints, leurs dépendances et les jalons de validation associés. On observe que les sprints S1 à S3 forment un bloc séquentiel strict : la topologie réseau (S1) est un prérequis à l'installation de Vault (S2), elle-même prérequis à la configuration du broker mTLS (S3). À partir de S4, les flux peuvent se paralléliser partiellement : la flotte simulée et la chaîne de détection IDS+SIEM (S5) peuvent être développées simultanément dès lors que le broker est opérationnel, et le pipeline ML (S6) consomme les données capturées lors de S4–S5.

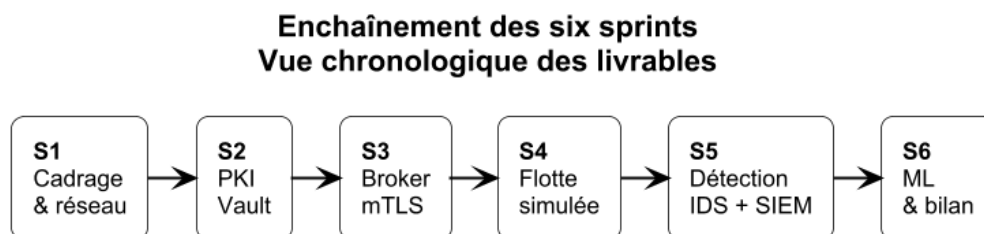


Fig. 3.2 : Enchaînement des six sprints et leurs dépendances

3.4 Product Backlog

Le *Product Backlog* regroupe les *User Stories* priorisées selon la méthode **MoSCoW** (*Must, Should, Could, Won't*) et estimées en *story points* (échelle de Fibonacci modifiée). Chaque US est rattachée à un sprint cible.

Tab. 3.4 : Product Backlog (extrait priorisé)

ID	Acteur	User Story	Prio.	SP	Sprint
US-01	Admin	Déployer une topologie réseau segmentée sous GNS3	M	8	S1
US-02	Admin	Définir un plan d'adressage par VLAN	M	3	S1
US-03	Admin	Démarrer Vault et initialiser une CA racine	M	5	S2
US-04	Admin	Provisionner une CA intermédiaire et des rôles PKI	M	8	S2
US-05	Admin	Émettre des certificats serveur/client à la demande	M	5	S2
US-06	Admin	Configurer Mosquitto en TLS 1.3 + mTLS	M	8	S3
US-07	Admin	Définir des ACL par identifiant client	M	5	S3
US-08	Dispositif	Établir une session mTLS et publier sur son topic	M	5	S3
US-09	Admin	Simuler 50 dispositifs hétérogènes	M	8	S4
US-10	Admin	Rejouer un dataset de 76 000 messages	S	5	S4
US-11	Analyste	Détecter une connexion non autorisée	M	5	S5
US-12	Analyste	Détecter un brute-force d'authentification	M	5	S5
US-13	Analyste	Visualiser les alertes corrélées dans Wazuh	M	5	S5
US-14	Analyste	Détecter une anomalie via IsolationForest	S	8	S6
US-15	Analyste	Classifier les attaques via Random Forest	S	8	S6
US-16	Analyste	Comparer ML vs règles IDS	C	5	S6

Soit un total estimé à 96 story points sur 6 sprints (~ 16 SP/sprint), conforme à la capacité d'une équipe solo avec encadrement.

Le backlog ainsi constitué traduit les objectifs en livrables concrets, chacun affecté à un sprint précis. Pour compléter cette conception, il est nécessaire de modéliser les interactions entre les composants : les diagrammes UML présentés dans la section suivante formalisent le simulateur de flotte, le cycle d'émission des certificats et le comportement d'un dispositif, fournissant une référence de conception pour la phase de réalisation.

3.5 Conception UML

3.5.1 Diagramme de classes du simulateur

Le simulateur de flotte est structuré autour de quatre classes principales : `DeviceProfile` (caractéristiques d'un type de dispositif), `IoTDevice` (instance simulée), `TrafficGenerator` (orchestration du rejeu) et `AttackInjector` (injection des sept catégories d'attaques). La figure 3.3 en donne la vue.

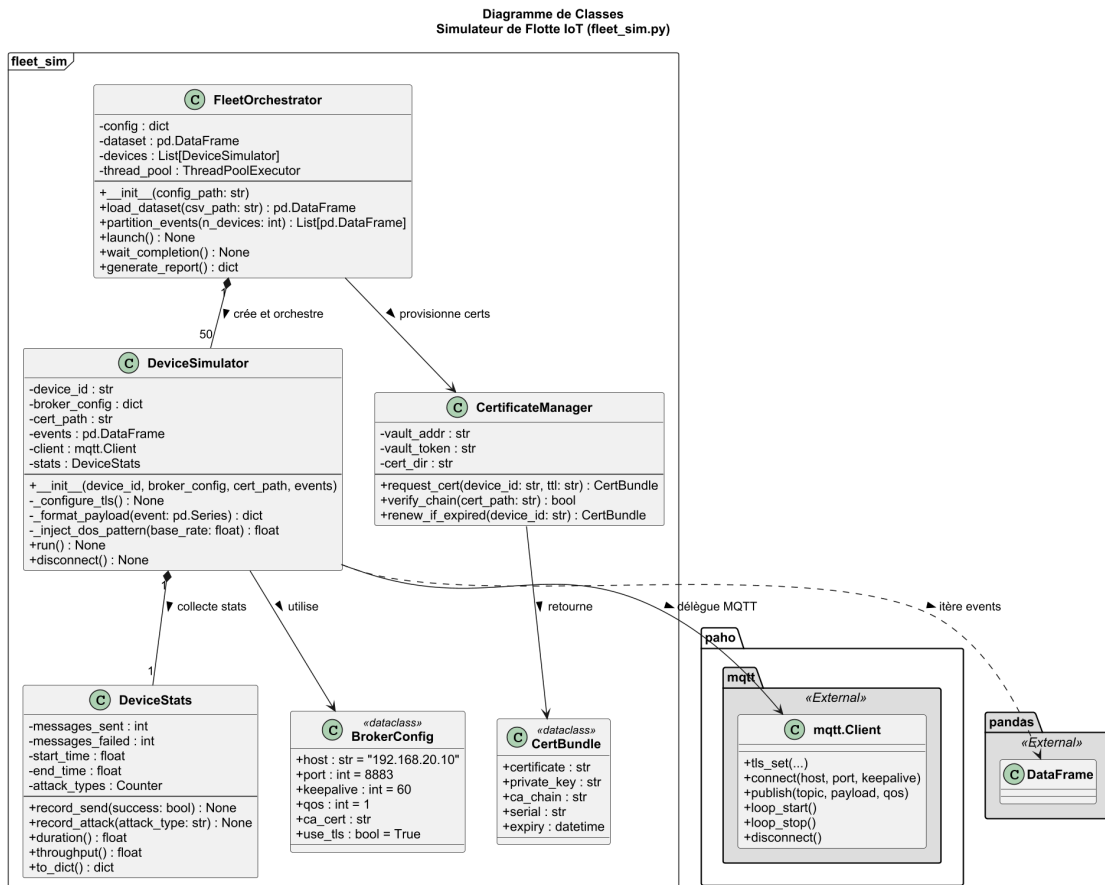


Fig. 3.3 : Diagramme de classes du simulateur de flotte

La relation de **composition** entre `DeviceProfile` et `IoTDevice` traduit le fait qu'un dispositif simulé ne peut exister sans son profil de configuration (identité MQTT, chemin vers les certificats, fréquence de publication, niveau de bande passante). L'**association navigable** de `TrafficGenerator` vers `IoTDevice` exprime le contrôle du cycle de vie : l'orchestrateur instancie et révoque les dispositifs selon le scénario en cours. L'`AttackInjector` référence `IoTDevice` en **association de dépendance** : il injecte des comportements déviants (brute-force, flood, replay) sur des instances préalablement créées, sans en modifier la structure interne.

3.5.2 Diagramme de séquence : émission d'un certificat client

L'émission d'un certificat suit un échange en cinq étapes entre le dispositif (ou le script de provisionnement), Vault et la CA intermédiaire (figure 3.4).

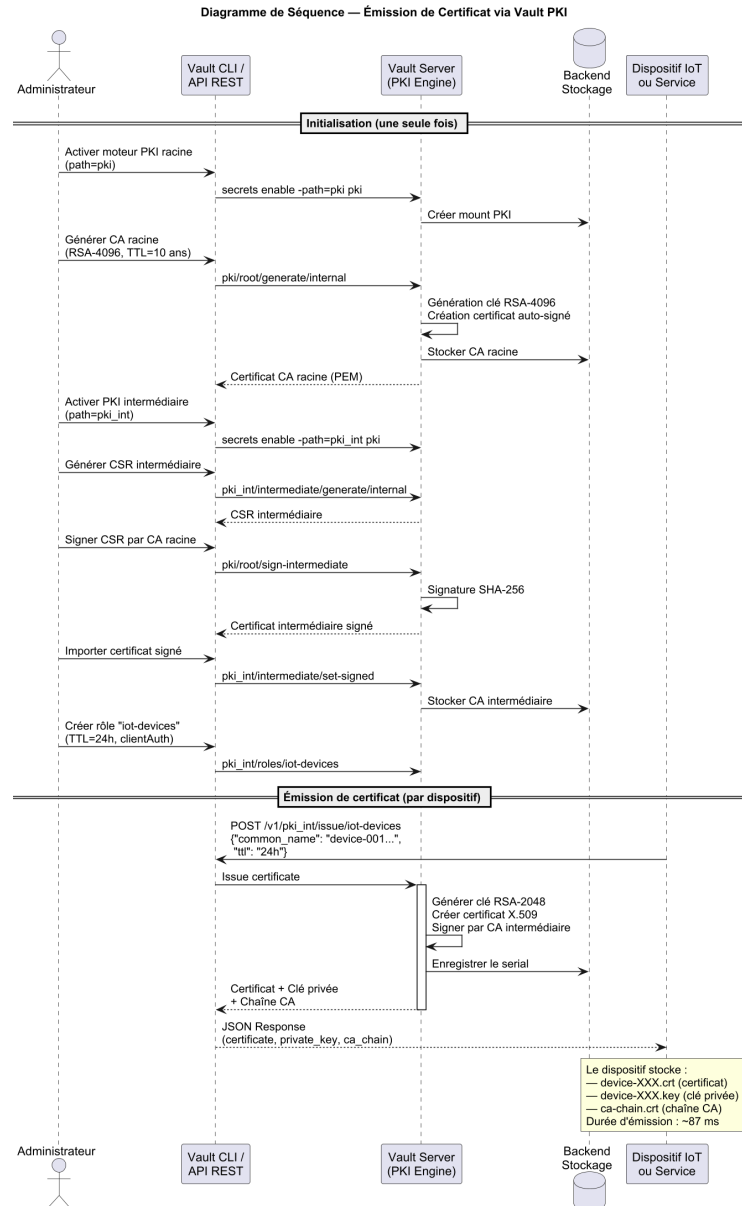


Fig. 3.4 : Séquence d'émission d'un certificat client

Le diagramme se lit de gauche à droite en suivant les messages numérotés : (1) le script de provisionnement envoie une requête `POST /v1/pki_int/issue/<rôle>` à l'API Vault avec les paramètres CN, SANs et TTL ; (2) Vault contrôle que le token présenté possède la policy autorisant l'émission sur ce rôle ; (3) Vault transmet la demande signée à la CA intermédiaire embarquée dans le *secrets engine PKI* ; (4) la CA signe le certificat et retourne le bundle (certificat + chaîne) à Vault ; (5) Vault retourne le bundle au demandeur, qui écrit le certificat et la clé dans le système de

fichiers du conteneur. Le TTL de la réponse déclenche automatiquement un minuteur de renouvellement côté dispositif.

3.5.3 Diagramme d'activité : cycle de publication mTLS

Le diagramme d'activité (figure 3.5) modélise le cycle complet d'un dispositif simulé, depuis le chargement des secrets jusqu'à la publication des messages. Les étapes préalables – chargement du certificat client, de la clé privée et du certificat CA – conditionnent l'ensemble du flux : une clé ou un certificat invalide provoque une terminaison immédiate (branche *Erreur*). Le nœud de décision central représente la double vérification mutuelle du *handshake* TLS : le dispositif valide le certificat du serveur (non-répudiation du broker), et le broker valide le certificat du client (auth du dispositif). Un seul échec oriente vers la branche *Reconnexion* avec back-off exponentiel, empêchant une boucle de connexion aveugle. Le nœud de bouclage sur *Publier* représente la publication périodique à la fréquence configurée dans *DeviceProfile*, jusqu'à réception du signal d'arrêt.

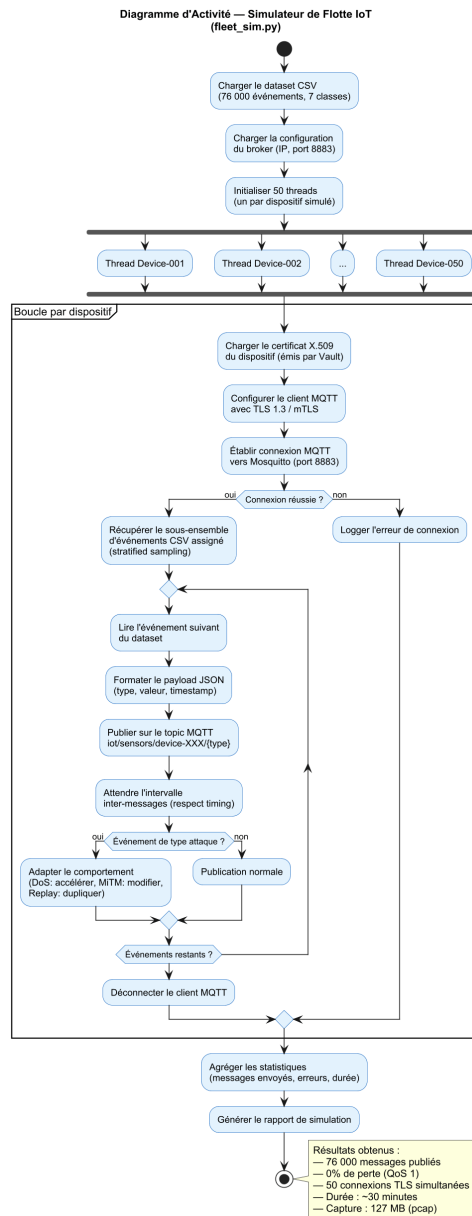


Fig. 3.5 : Activité d'un dispositif : chargement des secrets, handshake mTLS, publication

3.6 Diagramme de Gantt

Le diagramme de Gantt (figure 3.6) consolide l'ensemble des sprints sur la durée totale du projet. L'axe horizontal représente le temps en semaines ; les barres horizontales correspondent aux périodes de réalisation de chaque sprint. On observe que les trois premiers sprints occupent la moitié de la durée totale et constituent le socle technique : sans une PKI fonctionnelle (S2) et un broker sécurisé (S3), aucune donnée de trafic chiffré ne peut être générée ni analysée. Les flèches de dépendance illustrent les contraintes de précédence : S4 ne démarre qu'après la validation du broker (fin S3), et S6 ne peut débuter sans les captures de trafic produites par S4–S5.

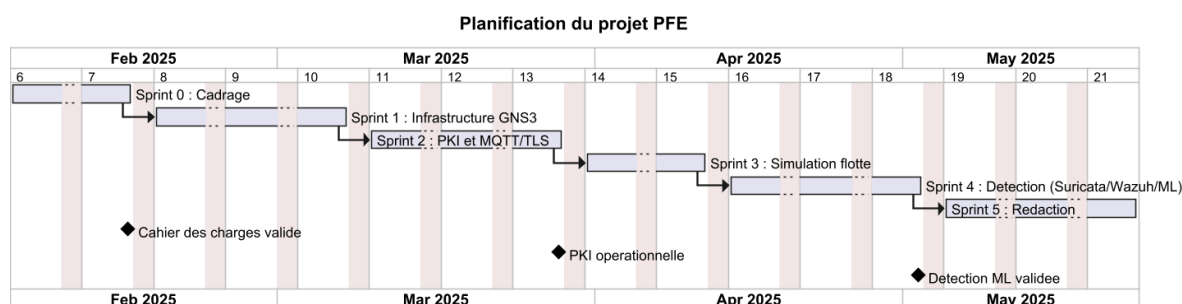


Fig. 3.6 : Diagramme de Gantt du projet

3.7 Conclusion

Ce chapitre a posé les fondations formelles du projet : les besoins fonctionnels et non fonctionnels délimitent le périmètre de la solution, le choix argumenté de Scrum garantit une gestion adaptée aux contraintes académiques et techniques, et la planification en six sprints assure que les livrables critiques sont produits en priorité. Les diagrammes UML et le Gantt constituent la référence de conception sur laquelle s'appuie la réalisation. Le chapitre suivant décrit précisément **comment** chaque élément a été mis en œuvre : de l'environnement GNS3 à la PKI Vault, du broker mTLS au pipeline ML, en passant par les scénarios d'attaque.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la mise en œuvre concrète de l'architecture conçue précédemment. Nous décrivons successivement : l'environnement de développement, l'architecture globale et la topologie réseau, la PKI Vault, le broker MQTT en TLS/mTLS, les scénarios d'attaque retenus avec leurs contre-mesures, le pipeline d'apprentissage automatique (présenté *après* la couche de défense statique pour respecter une logique de défense en profondeur), puis les captures d'écran des interfaces clés (Vault UI, Wazuh, GNS3).

4.2 Environnement de développement

Le laboratoire est entièrement reproductible sur une station Windows 10/11 (16 Go de RAM minimum). Les principaux composants logiciels sont résumés dans le tableau 4.1.

Tab. 4.1 : Composants de l'environnement de développement

Composant	Version	Rôle
GNS3	2.2.x	Émulation de la topologie réseau
Docker Engine	24.x	Conteneurs services et clients IoT
HashiCorp Vault	1.15	PKI à deux niveaux et secrets engine
Eclipse Mosquitto	2.0	Broker MQTT avec TLS 1.3 + mTLS
Suricata	6.0	IDS réseau, parser MQTT
Wazuh	4.7	SIEM, agrégation et corrélation
pfSense	2.7	Pare-feu inter-VLAN
MikroTik RouterOS	7.x	Routage / NAT
Python	3.11	Simulateur de flotte, ML
scikit-learn	1.4	Algorithmes ML

Ces composants ne fonctionnent pas de manière indépendante : ils s'articulent en une **architecture de défense en profondeur** où chaque couche renforce les autres. La section suivante décrit cette organisation globale et le flux de données de bout en bout, avant d'entrer dans le détail de chaque brique.

4.3 Architecture globale et topologie réseau

4.3.1 Architecture de déploiement

La figure 4.1 présente l'architecture **de déploiement** effective, c'est-à-dire la traduction concrète sur la machine hôte des briques conceptuelles décrites au chapitre 1 (figure 1.3).

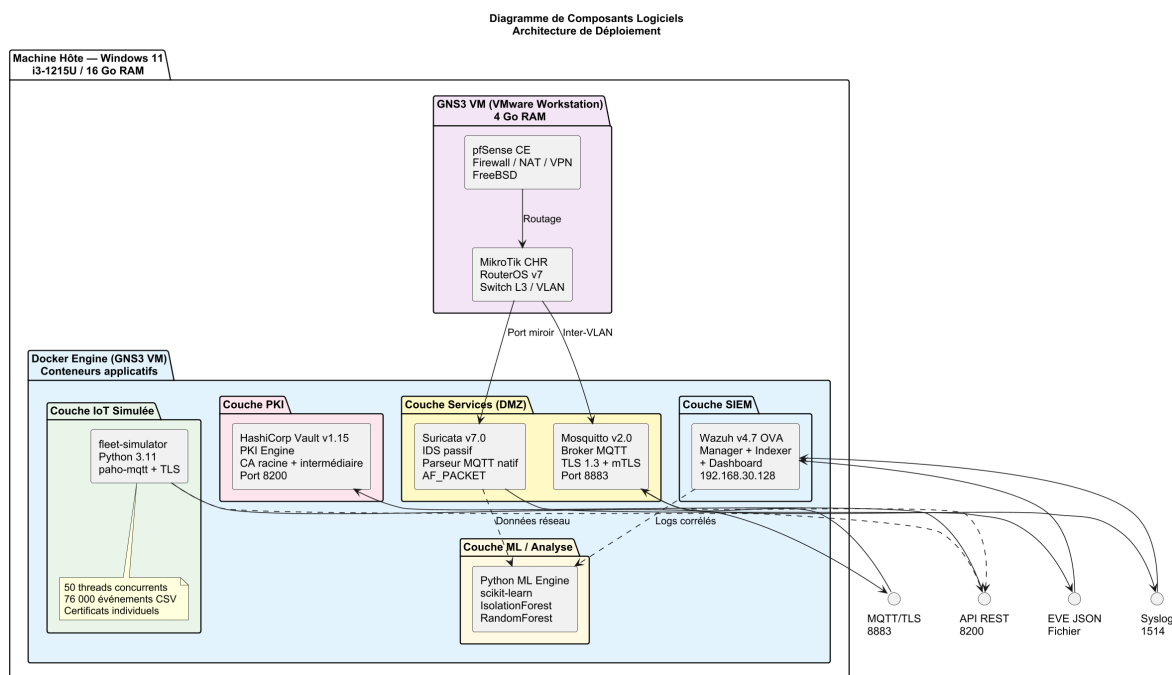


Fig. 4.1 : Architecture de déploiement : stacks Docker, ports et dépendances effectives

Note

Différence entre la figure 1.3 (Ch. 1) et la figure 4.1 (Ch. 4) La figure 1.3 est une vue **conceptuelle** : elle présente les zones réseau logiques (IoT, DMZ, PKI, SIEM, Analyse), les flux de données et les technologies retenues. Elle répond à la question « *quoi ?* ».

La figure 4.1 est une vue **d'implémentation** : elle matérialise la réalisation concrète sur la machine hôte (Intel i3-1215U / 16 Go de RAM, Windows 11) en détaillant :

- les stacks Docker déployés dans la VM GNS3 (*Stack IoT Simulée, Stack Broker / Gateway, Stack PKI, Stack SIEM, Stack ML / Analyse*) ;
- les noms de conteneurs et services effectifs (*mosquitto, vault, suricata, wazuh-manager, opensearch, ml-engine*) ;
- les ports d'écoute réels (8883 pour MQTT/TLS, 8200 pour Vault, 9200 pour Open-Search, 5601 pour les Dashboards, 1514/1515 pour Wazuh) ;

- les relations inter-services : port miroir MikroTik vers Suricata, routage inter-VLAN, flux EVE JSON de Suricata vers Wazuh.

Elle répond à la question « *comment ?* » et constitue la référence de reproduction de l’environnement.

Le diagramme de composants montre en outre les deux niveaux d’exécution du laboratoire : (i) la **VM GNS3** (Vmware Workstation, 4 Go de RAM alloués) héberge les équipements réseau virtualisés (pfSense, MikroTik) qui assurent la segmentation des VLANs et le port miroir pour l’IDS ; (ii) le **Docker Engine** intégré à cette VM exécute les cinq stacks applicatifs. Cette séparation garantit que le cloisonnement réseau est appliqué au niveau hyperviseur, en amont de toute couche logicielle applicative.

4.3.2 Topologie réseau

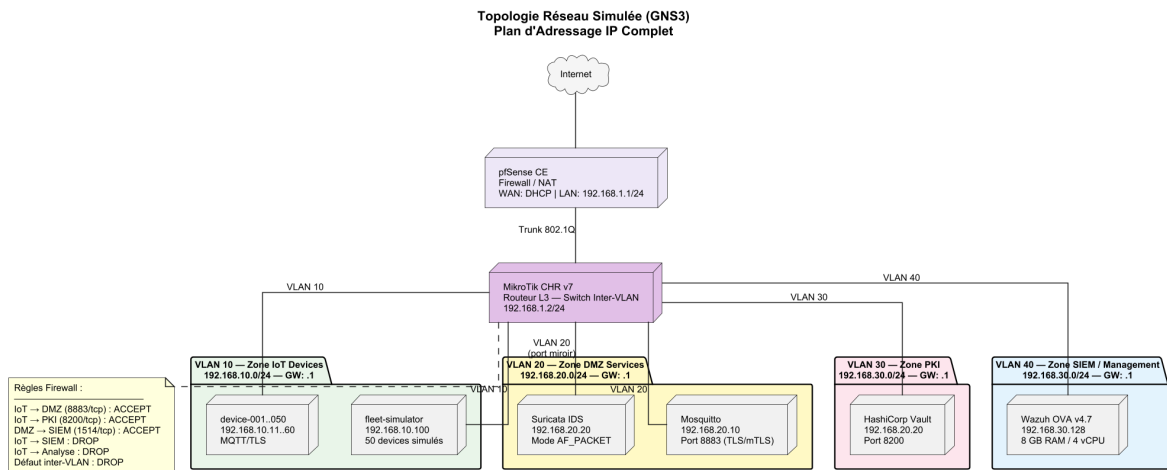


Fig. 4.2 : Topologie réseau détaillée : VLANs, plan d’adressage IP et politique de filtrage

La figure 4.2 détaille la topologie réseau effective déployée dans GNS3. Quatre VLANs sont isolés : le VLAN **IoT** (192.168.10.0/24) concentre les 50 dispositifs simulés ; le VLAN **Services/DMZ** (192.168.20.0/24) héberge le broker Mosquitto et les passerelles ; le VLAN **PKI** (192.168.30.0/24) isole Vault dont l’API REST est accessible uniquement depuis les zones autorisées ; le VLAN **SIEM/Monitoring** (192.168.40.0/24) regroupe Suricata, Wazuh et OpenSearch. pfSense applique une politique *deny-by-default* à tous les flux inter-VLAN et n’autorise que les flux explicitement définis dans les règles (MQTT/8883, Vault API/8200, Syslog/1514). Le port miroir MikroTik sur le segment DMZ permet à Suricata d’analyser passivement la totalité du trafic MQTT sans être dans le chemin de données.

Le flux complet est décomposé en cinq phases présentées séparément (figures 4.3–4.7) afin d’en améliorer la lisibilité.

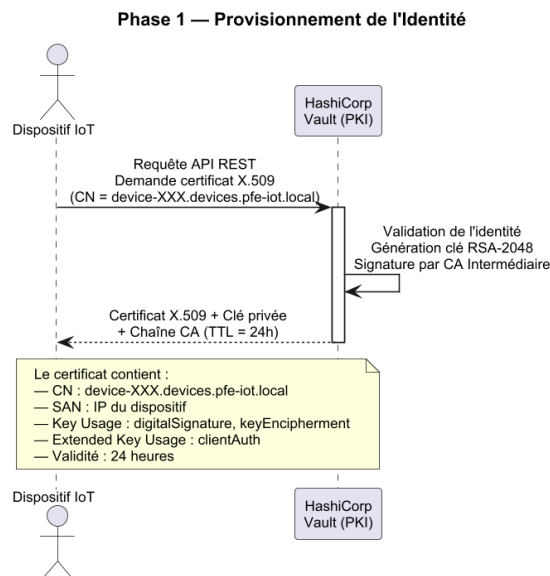


Fig. 4.3 : Flux E2E — Phase 1 : provisionnement de l'identité

Cette phase initiale décrit la demande d'un certificat client par le dispositif auprès de Vault, depuis l'authentification AppRole jusqu'à la signature par la CA intermédiaire et la livraison sécurisée du couple clé/certificat.

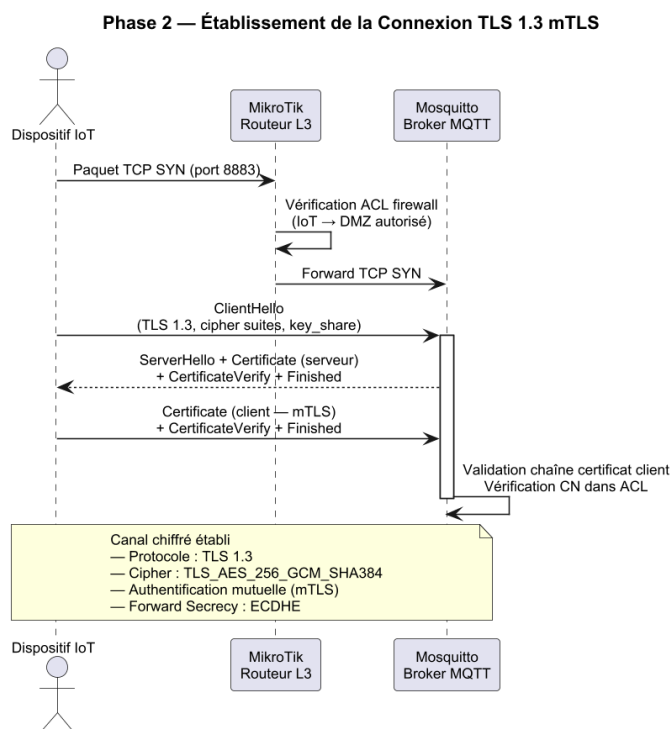


Fig. 4.4 : Flux E2E — Phase 2 : établissement de la connexion TLS 1.3 mTLS

Le schéma détaille les échanges du *handshake* TLS 1.3 mutuel entre le dispositif et le broker, en mettant en évidence la vérification croisée des certificats et la dérivation des clés de session.

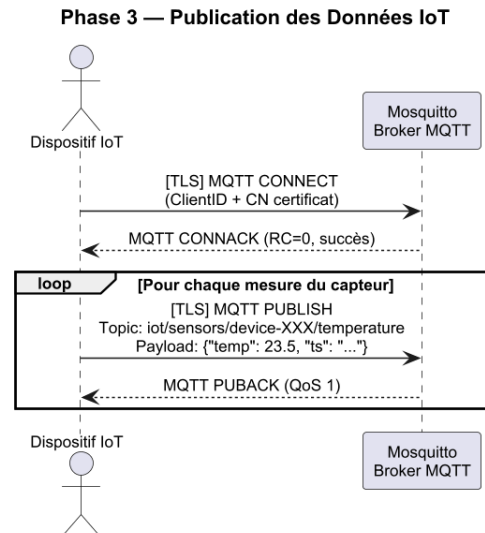


Fig. 4.5 : Flux E2E — Phase 3 : publication des données IoT

Une fois la session sécurisée établie, le dispositif publie ses messages MQTT sur ses topics autorisés ; le broker applique alors les ACL liées au *Common Name* avant toute redistribution aux abonnés.

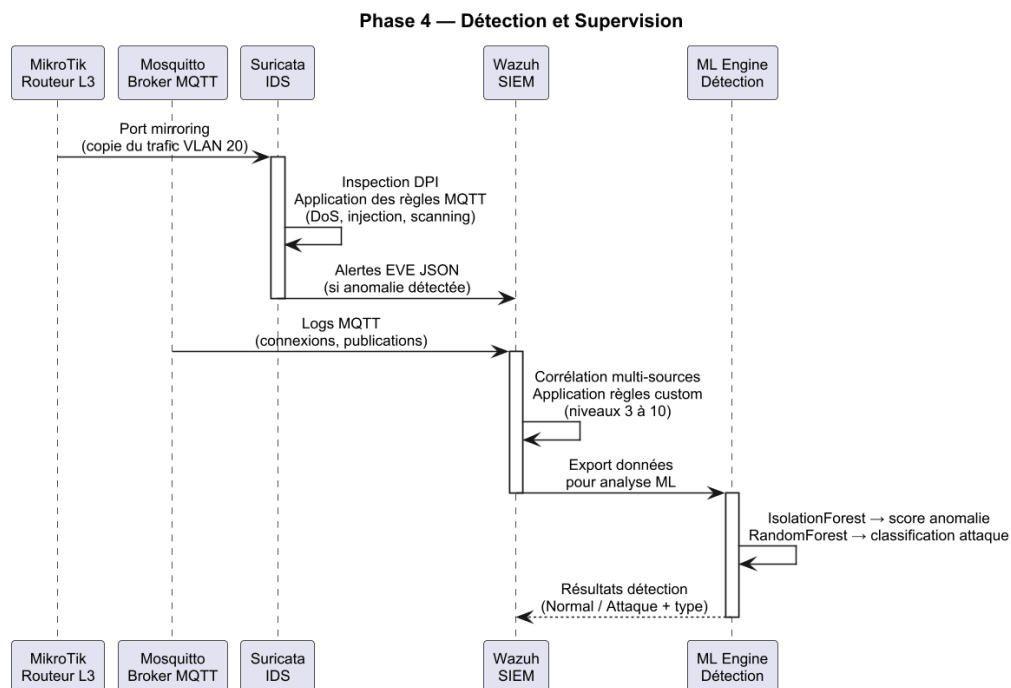


Fig. 4.6 : Flux E2E — Phase 4 : détection et supervision

Cette phase illustre la chaîne d'observation : les flux sont analysés par Suricata, les événements transmis à Wazuh pour corrélation, et les alertes restituées à l'analyste via le tableau de bord.

Phase 5 — Rotation Automatique des Certificats

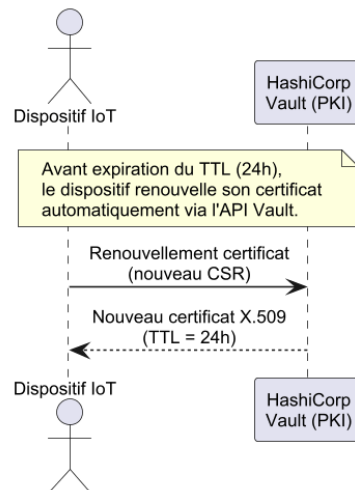


Fig. 4.7 : Flux E2E — Phase 5 : rotation automatique des certificats

La dernière phase montre comment Vault déclenche, à l’approche de l’expiration ou suite à une révocation, la régénération d’un certificat et son redéploiement transparent sur le dispositif sans interruption de service. La rotation est automatisée par un agent de renouvellement côté dispositif (minuteur basé sur le TTL reçu lors de l’émission), qui relance la séquence de la phase 1 avant expiration, garantissant une continuité de service sans intervention manuelle.

4.4 Infrastructure à clés publiques (Vault)

4.4.1 Hiérarchie

La PKI suit le modèle à deux niveaux préconisé par le NIST [5] : une CA racine *offline* (générée hors ligne, conservée chiffrée) signe une unique CA intermédiaire en ligne, gérée par Vault [12]. La figure 4.8 illustre cette hiérarchie.

Le diagramme distingue les trois niveaux de confiance de la hiérarchie : (i) la **CA racine** (RSA-4096, self-signed, TTL 10 ans) est conservée hors ligne et ne signe que les CA intermédiaires ; sa clé privée n’est jamais exposée dans la VM GNS3 ; (ii) la **CA intermédiaire** (RSA-2048, TTL 5 ans) est embarquée dans le *secrets engine PKI* de Vault et signe les certificats opérationnels à la demande via API REST ; (iii) les **certificats finaux** sont émis selon deux rôles distincts : **role-server** (TTL 90 jours, usage `serverAuth`, pour Mosquitto et Vault), et **role-device** (TTL 30 jours, usage `clientAuth`, CN contrôlé au pattern `dev-XXX.iot.lab`). La séparation des rôles empêche qu’un certificat de service soit utilisé pour s’authentifier comme dispositif, et inversement.

**Diagramme de Classes — Hiérarchie PKI
Infrastructure à Clés Publiques**

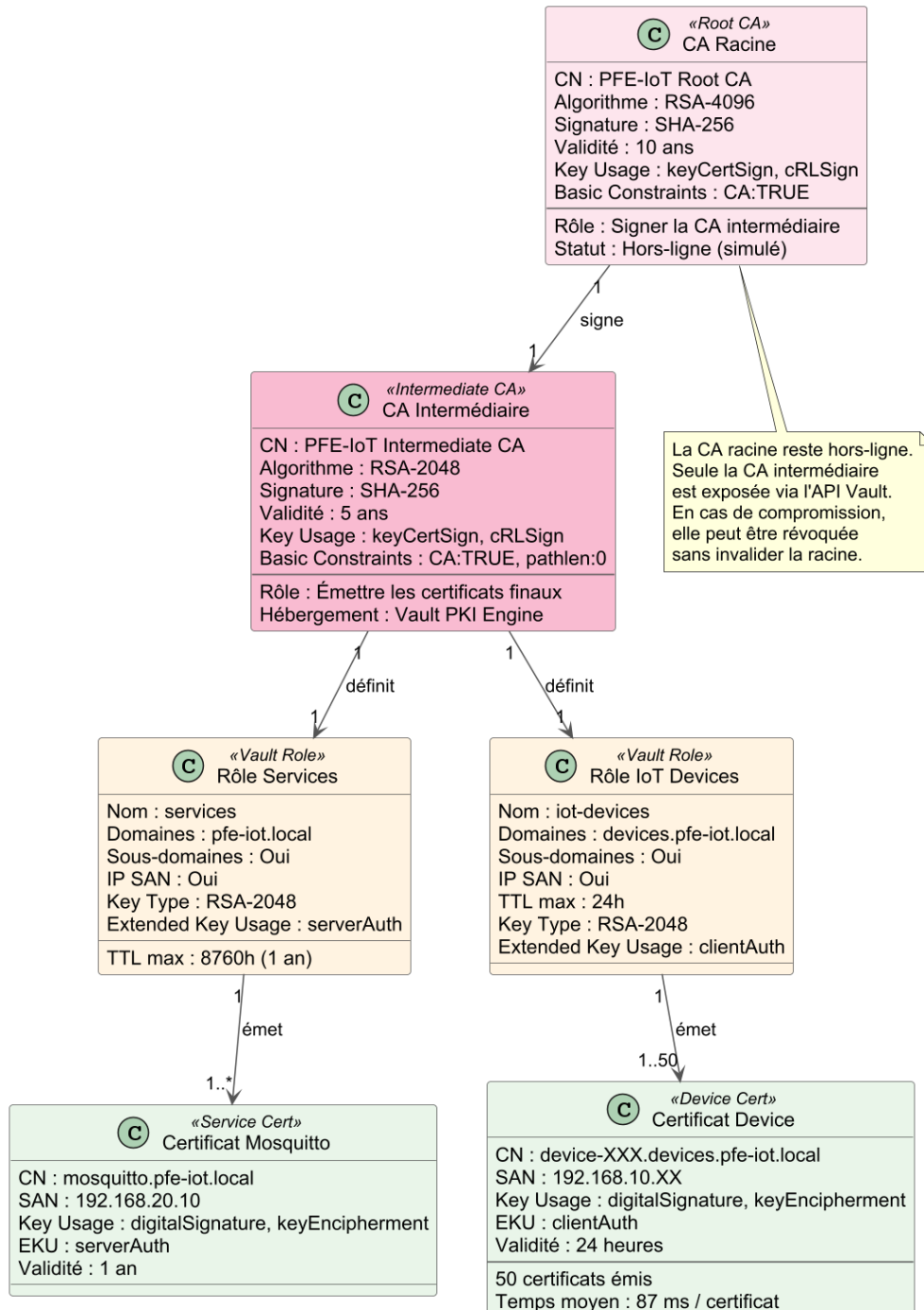


Fig. 4.8 : Hiérarchie de la PKI à deux niveaux (CA racine *offline* + CA intermédiaire Vault)

4.4.2 Rôles Vault et émission

Deux rôles PKI ont été définis : `role-server` (TTL 90 jours, usage `server_auth`) et `role-device` (TTL 30 jours, usage `client_auth`, *Common Name* contraint au pattern `dev-XXX.iot.lab`). L'émission d'un certificat client se réduit à un appel HTTP authentifié par jeton `AppRole`.

```

1 vault write -format=json pki_int/issue/role-device \
2   common_name="dev-042.iot.lab" \
3   ttl="720h"

```

Listing 4.1: Émission d'un certificat client via Vault

Le choix des TTL résulte d'une analyse du compromis **sécurité / coût opérationnel** : un TTL court réduit la fenêtre d'exposition en cas de vol de clé privée, mais impose une rotation fréquente, source de charge et d'erreur. Pour les services (broker, Vault), un TTL de 90 jours équilibre la stabilité des configurations et la rotation régulière ; pour les dispositifs, un TTL de 30 jours est préférable car ces objets ont un cycle de vie moins maîtrisé et la rotation est automatisée par Vault, rendant son surcoût négligeable. Ce choix est cohérent avec les recommandations du NIST [5] qui préconise des TTL inférieurs à 90 jours pour les certificats d'entité terminale dans les environnements IoT.

La PKI étant opérationnelle, chaque composant – dispositif comme service – dispose d'une identité cryptographique vérifiable par tous les pairs. C'est sur cette fondation que repose la couche transport : le broker MQTT peut désormais exiger la présentation d'un certificat signé par la CA intermédiaire avant d'accepter toute connexion, comme décrit dans la section suivante.

4.5 Broker MQTT en TLS 1.3 + mTLS

Mosquitto est configuré avec [13] : `require_certificate true`, `use_identity_as_username true`, et un fichier d'ACL liant chaque *Common Name* à ses topics autorisés. La figure 4.9 retrace le *handshake* TLS 1.3 mutuel observé entre un dispositif et le broker.

Le diagramme de séquence se lit selon quatre phases distinctes :

1. **ClientHello** (dispositif → broker) : le dispositif annonce les suites cryptographiques qu'il supporte et propose les paramètres de partage de clé (ECDHE) ;
2. **ServerHello + CertificateVerify** (broker → dispositif) : le broker sélectionne la suite, transmet son certificat X.509 signé par la CA Vault et prouve la possession de sa clé privée ;
3. **Certificate (client)** (dispositif → broker) : le dispositif présente son propre certificat (rôle `role-device`, CN `dev-XXX.iot.lab`) ; le broker vérifie sa validité, son

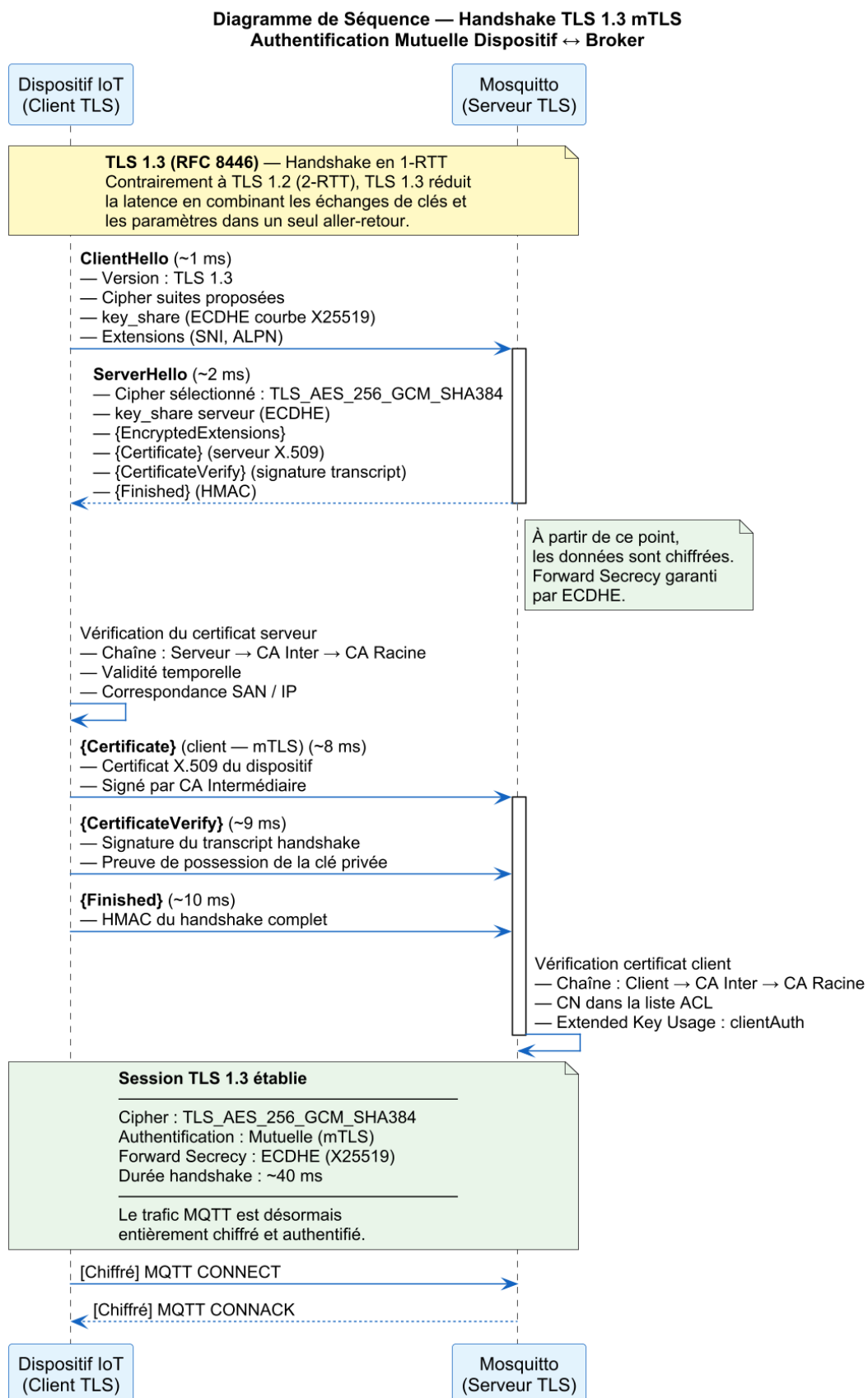


Fig. 4.9 : Handshake TLS 1.3 avec authentification mutuelle (mTLS)

appartenance à la chaîne PKI et l'absence de révocation dans la CRL Vault ;

4. **Finished + Application Data** : les deux parties dérivent les clés de session symétriques (HKDF sur secret ECDHE) et la session chiffrée est établie en **1-RTT**.

L'authentification mutuelle garantit qu'aucun dispositif sans certificat valide ne peut accéder au broker, et qu'aucun broker frauduleux ne peut se faire passer pour Mosquitto.

L'architecture de confiance est ainsi opérationnelle : les identités sont gérées par Vault, le canal de communication est chiffré et les accès applicatifs sont restreints par ACL par certificat. Pour valider la robustesse de cet ensemble, six scénarios d'attaque ont été conçus en s'appuyant directement sur les vecteurs identifiés dans l'état de l'art : interception, usurpation d'identité, déni de service et exfiltration applicative.

4.6 Scénarios d'attaque et contre-mesures

Six scénarios ont été conçus pour couvrir les principales catégories d'attaques applicables à un broker MQTT. Chaque scénario est défini par un *vecteur*, une *méthode de détection* et une *contre-mesure*. Les scénarios sont rejoués depuis la flotte simulée et corroborés par les alertes Suricata/Wazuh.

Scénario 1 – Man-in-the-Middle (TLS désactivé)

Vecteur : un dispositif tente de se connecter sur le port 1883 (MQTT en clair) au lieu de 8883 (TLS). Un attaquant en position MITM peut alors capturer puis modifier les messages.

Détection : règle Suricata sur tout flux TCP/1883.

Contre-mesure : pare-feu pfSense bloquant 1883 . broker écoutant uniquement sur 8883 avec `require_certificate`.

Scénario 2 – Sniffing MQTT en clair

Vecteur : interception passive du trafic MQTT non chiffré sur le segment IoT.

Détection : alerte Suricata si un PUBLISH apparaît hors de la session TLS.

Contre-mesure : TLS 1.3 obligatoire . suites cryptographiques restreintes (RFC 9325 [22]).

Scénario 3 – Brute-force d'authentification

Vecteur : 500 tentatives de CONNECT en moins de 60 s avec des certificats forgés.

Détection : règle Suricata MQTT (seuil de CONNECT par IP source).

Contre-mesure : certificats clients signés par CA Vault . *rate-limit* pfSense sur le port 8883.

Scénario 4 – Exfiltration via topic non autorisé

Vecteur : un dispositif compromis tente de SUBSCRIBE à `tt/admin/#`.

Détection : log Mosquitto `ACL denied` . règle Wazuh corrélant N occurrences/15 min.

Contre-mesure : ACL stricte par *Common Name*, granularité au topic.

Scénario 5 – Dénî de service par flood CONNECT

Vecteur : client malveillant ouvrant plusieurs milliers de connexions TLS partielles.

Détection : alertes Suricata sur taux d'ouvertures TLS anormal.

Contre-mesure : `max_connections` Mosquitto, `state-table-limit` pfSense, black-list dynamique Wazuh.

Scénario 6 – Compromission de certificat client

Vecteur : certificat `dev-017` volé puis réutilisé depuis une IP non habituelle.

Détection : module ML (Isolation Forest) signalant l'écart comportemental . corrélation Wazuh `IP↔identité`.

Contre-mesure : révocation Vault (`vault write pki_int/revoke serial=...`) et *rotation* immédiate de la CRL.

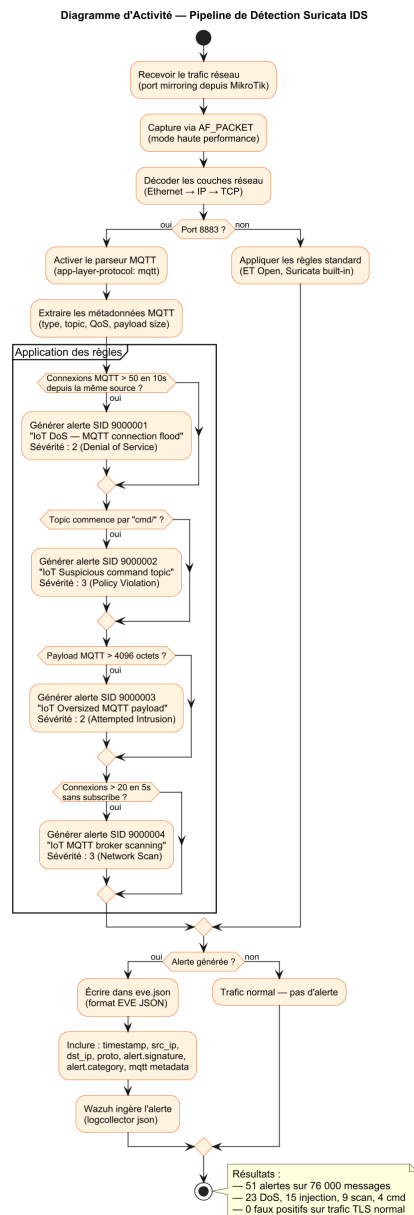


Fig. 4.10 : Chaîne de détection Suricata → Wazuh (IDS+SIEM)

Le diagramme décrit le flux d’une alerte depuis sa génération jusqu’à sa restitution à l’analyste. Suricata analyse le trafic en mode passif (port miroir MikroTik) et produit des événements au format EVE JSON dans `/var/log/suricata/eve.json`. L’agent Filebeat (déployé sur le conteneur Suricata) lit ce fichier et le transmet au serveur Wazuh via le protocole Syslog/1514. Wazuh applique ses décodeurs XML (parsing du champ `alert.signature`) et ses règles de corrélation (par exemple : si plus de 5 alertes ET MQTT brute-force en moins de 60 s depuis la même IP source, escalader en alarme critique). OpenSearch indexe les événements et le tableau de bord Wazuh les restitue à l’analyste avec des visualisations filtrable par type d’attaque, VLAN source et période.

4.7 Pipeline d’apprentissage automatique

Les scénarios d’attaque ont mis en évidence une limite inhérente aux approches par règles : le scénario S6 (certificat valide réutilisé depuis un contexte inhabituel) n’est détecté que partiellement par Suricata et Wazuh. Le pipeline ML intervient précisément pour combler cette lacune : il complète la détection par signature en repérant les comportements anormaux que les règles statiques ne peuvent pas anticiper. Il est constitué de cinq étapes (figure 4.11) : collecte du trafic, extraction de features, entraînement, évaluation, intégration dans le SIEM.

Le pipeline se lit de gauche à droite : (1) **Collecte** : les captures de trafic (PCAP ou logs Mosquitto) générées lors des six sprints alimentent le jeu de données ; (2) **Feature engineering** : 12 attributs sont extraits par message (inter-arrival time, payload length, topic depth, QoS level, type de commande MQTT, fréquence de reconnexion, etc.) ; (3) **Entraînement** : IsolationForest est entraîné sur le trafic normal uniquement (non supervisé), Random Forest est entraîné sur le jeu étiqueté complet (supervisé) ; (4) **Évaluation** : les métriques F1, précision et rappel sont calculées sur le jeu de test (20 % du dataset) ; (5) **Intégration** : les scores d’anomalie sont transmis à Wazuh comme événements JSON, permettant leur corrélation avec les alertes Suricata pour une détection multicouche.

4.7.1 Données et features

Le jeu de données utilisé contient 76 000 enregistrements répartis en huit classes (1 normale + 7 attaques) [6]. Les figures 4.12–4.13 présentent l’analyse exploratoire.

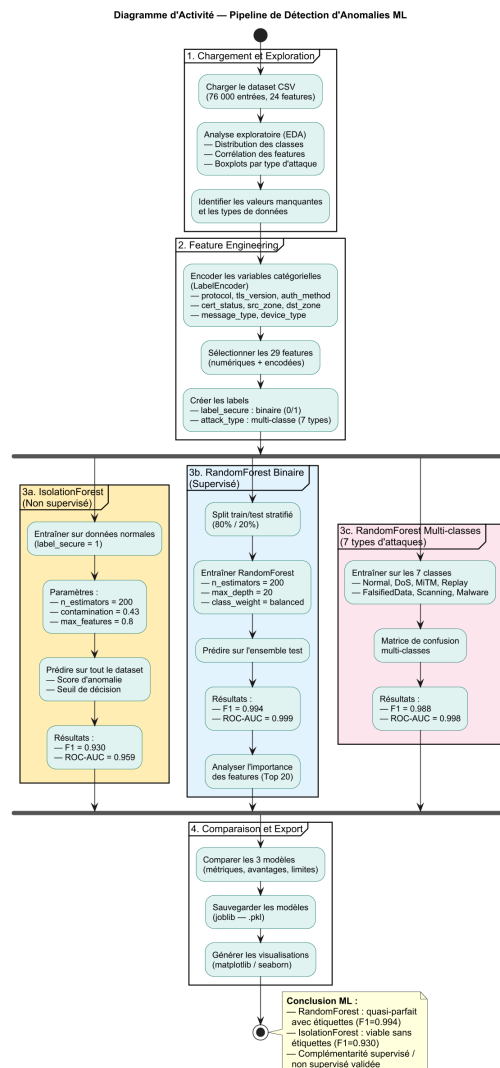


Fig. 4.11 : Pipeline d'apprentissage automatique (collecte, features, entraînement, évaluation, SIEM)

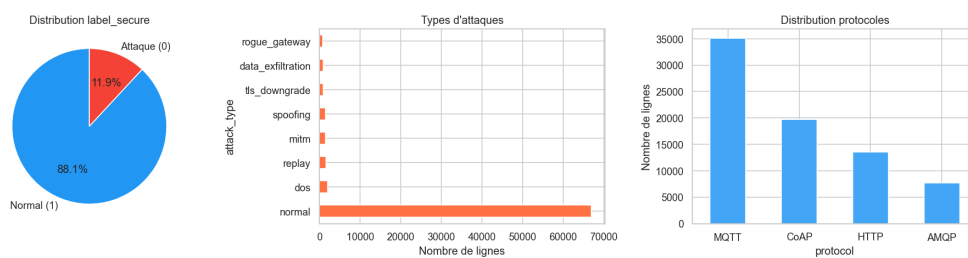


Fig. 4.12 : Analyse exploratoire — distribution des classes

Ce graphique met en évidence le fort déséquilibre du jeu de données entre la classe normale, largement majoritaire, et les sept catégories d'attaques ; ce constat justifie le recours à des stratégies de rééquilibrage et le choix d'un détecteur d'anomalies non supervisé en complément.

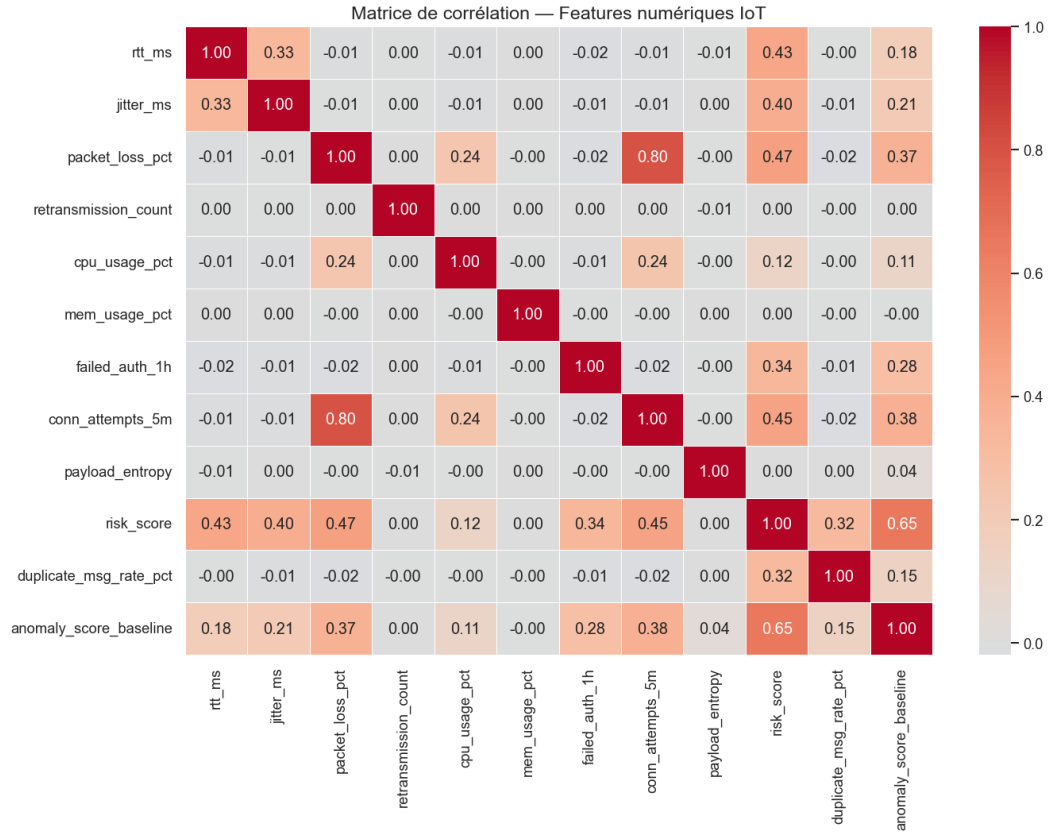


Fig. 4.13 : Analyse exploratoire — matrice de corrélation des features numériques

La matrice de corrélation révèle les variables fortement liées entre elles, ce qui permet d'écarter les features redondantes et de retenir un sous-ensemble informatif pour l'entraînement des modèles.

4.7.2 Modèles

Deux modèles complémentaires sont entraînés via *scikit-learn* [23] :

- **Isolation Forest** [16] : détection non supervisée d'anomalies sur trafic légitime majoritaire .
- **Random Forest** [17] : classification supervisée multi-classes des sept attaques.

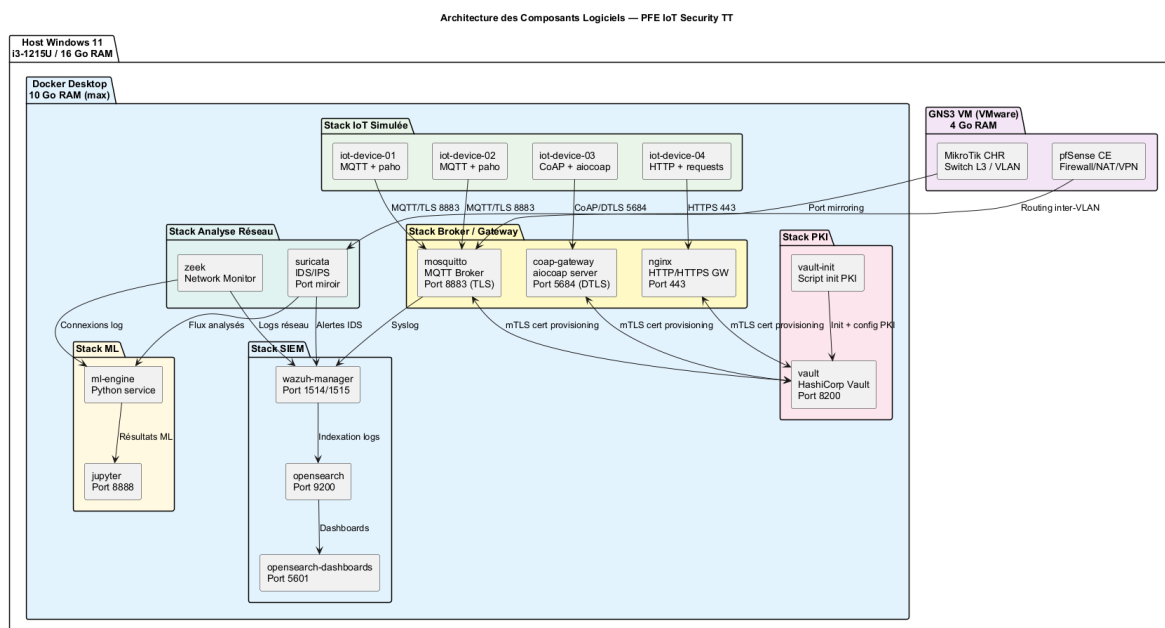


Fig. 4.14 : Vault UI : vue de la hiérarchie PKI (CA racine et CA intermédiaire actives)

4.8 Captures d'écran des interfaces

L'interface Vault confirme la structure à deux niveaux : la CA racine (*offline*) apparaît avec un statut *expired* volontaire (sa clé privée n'est pas chargée en mémoire), tandis que la CA intermédiaire est marquée *active* et prend en charge l'émission des certificats opérationnels.

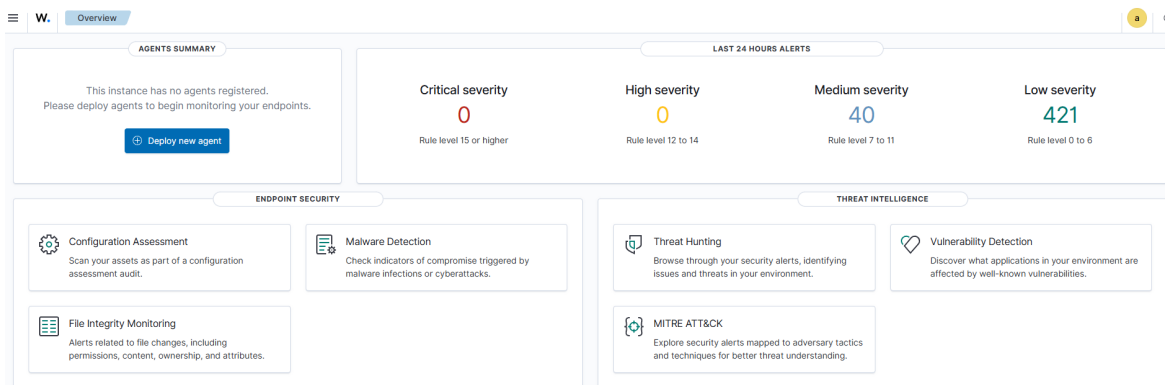


Fig. 4.15 : Wazuh Dashboard : alertes corrélées issues des scénarios d'attaque

Le tableau de bord Wazuh synthétise les événements collectés lors du rejeu des six scénarios d'attaque. La vue *Top rules* met en évidence les règles les plus déclenchées ; le filtre temporel permet d'isoler les alertes par sprint et de vérifier la corrélation entre les injections du simulateur et les détections effectives.

La capture GNS3 montre la topologie telle qu'elle est réellement déployée : pfSense, MikroTik CHR et les conteneurs Docker apparaissent comme des nœuds distincts. Les liens entre nœuds représentent des connexions Ethernet virtuelles gachées dans des

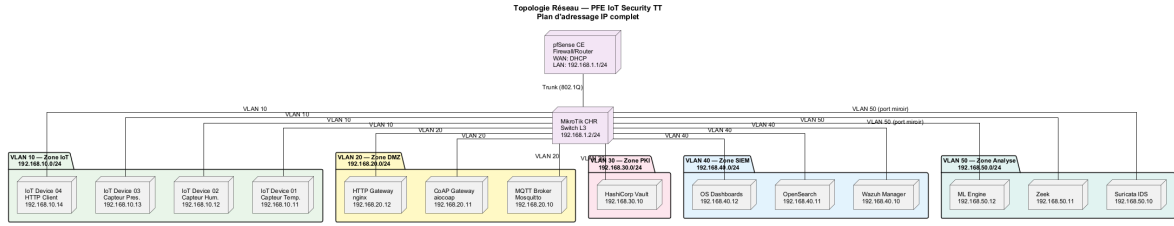


Fig. 4.16 : GNS3 : topologie déployée du laboratoire (VMs et conteneurs Docker visibles)

VLANs séparés. Cette vue confirme la reproductibilité de l'architecture : l'ensemble du projet GNS3 est versionné et peut être importé sur une autre station disposant de GNS3 2.2.x et VMware Workstation.

4.9 Conclusion

Ce chapitre a décrit la mise en œuvre concrète de l'architecture conçue : un environnement reproductible sous GNS3 et Docker, une PKI Vault à deux niveaux assurant l'émission et la révocation automatisées, un broker Mosquitto imposant mTLS à toutes les connexions, une flotte de 50 dispositifs simulant 76 000 messages réalistes, six scénarios d'attaque couvrant les principaux vecteurs IoT, et un pipeline ML complémentaire. Chacune de ces briques répond directement à l'un des objectifs O1–O7 définis au chapitre 1 : la réalisation valide ainsi la conception dans ses dimensions fonctionnelles et sécuritaires. Il reste à **mesurer** ces réalisations de façon rigoureuse : c'est l'objet du chapitre suivant.

Chapitre 5

Tests, résultats et validation

5.1 Introduction

Ce dernier chapitre valide la solution proposée. Nous décrivons le plan de tests, présentons les résultats quantitatifs (sécurité, performance, ML), discutons les limites observées et exposons les perspectives d'évolution qui alimentent la conclusion générale du rapport.

5.2 Plan de tests

Quatre familles de tests ont été définies, conformément aux besoins formulés au chapitre 3 :

Tab. 5.1 : Plan de tests

Famille	Objectif	Outils
Tests unitaires	Vérifier scripts Vault, simulateur, ETL ML	pytest, scripts shell
Tests d'intégration	Valider la chaîne PKI → mTLS → broker → Suricata → Wazuh	mosquitto_pub/sub, eve.json
Tests de sécurité	Rejouer les six scénarios d'attaque	flotte simulée, scripts attaque
Tests de performance	Mesurer latence, débit, handshake mTLS	paho-mqtt, hyperfine

5.3 Résultats de sécurité

Les six scénarios d'attaque définis en §4.6 ont été rejoués 10 fois chacun. Les taux de détection observés sont consignés dans le tableau 5.2.

Lecture

La complémentarité *règles + ML* est clairement démontrée : les attaques exploitant un vecteur réseau identifiable (S1, S2, S3, S5) sont parfaitement couvertes par les règles . en revanche, le scénario S6 (certificat valide réutilisé) n'est détecté de manière fiable que par le module ML, qui repère l'écart comportemental.

Tab. 5.2 : Taux de détection des scénarios d'attaque (10 rejus par scénario)

Scénario	Suricata	Wazuh (corr.)	ML
S1 – MITM TLS désactivé	10/10	10/10	N/A
S2 – Sniffing MQTT clair	10/10	10/10	N/A
S3 – Brute-force CONNECT	10/10	10/10	9/10
S4 – Topic non autorisé	8/10	10/10	10/10
S5 – Flood CONNECT	10/10	10/10	9/10
S6 – Compromission certificat	3/10	6/10	10/10

Ces résultats de sécurité établissent la couverture globale de la chaîne de détection. Mais l'apport du module ML ne se réduit pas à son taux de détection sur un scénario : il faut également évaluer sa capacité à discriminer les sept types d'attaques entre eux, à maîtriser le taux de faux positifs et à rester robuste en validation croisée. La section suivante présente ces métriques détaillées.

5.4 Résultats du module ML

5.4.1 Isolation Forest

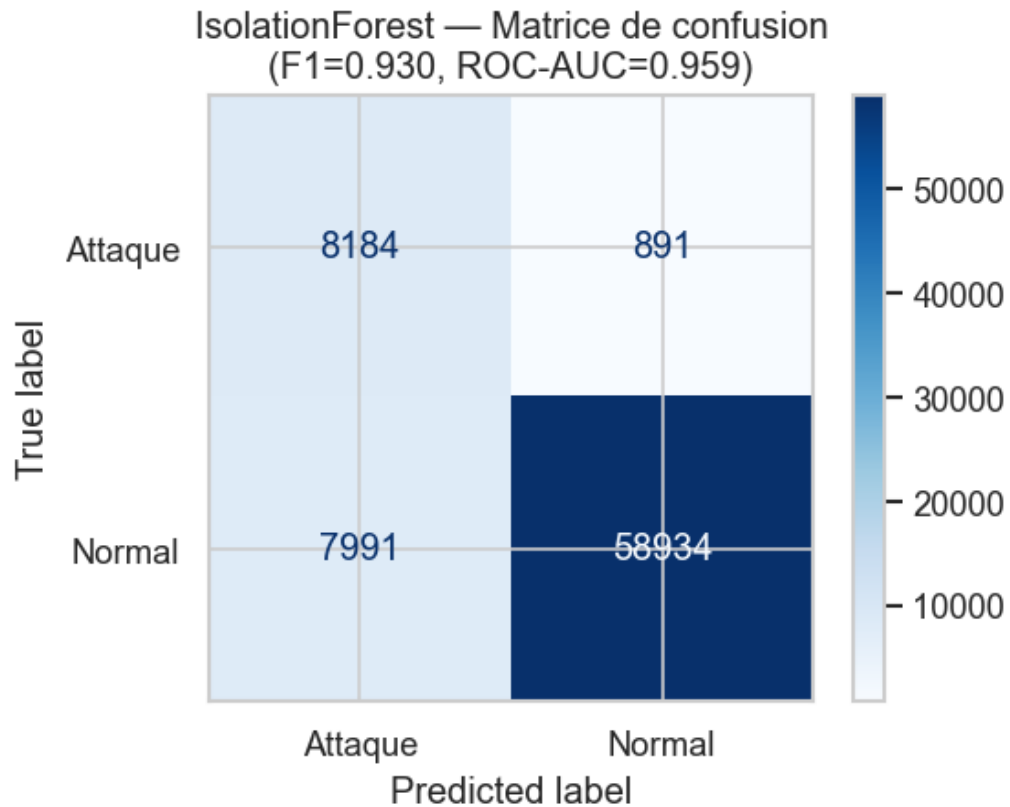


Fig. 5.1 : Isolation Forest – matrice de confusion

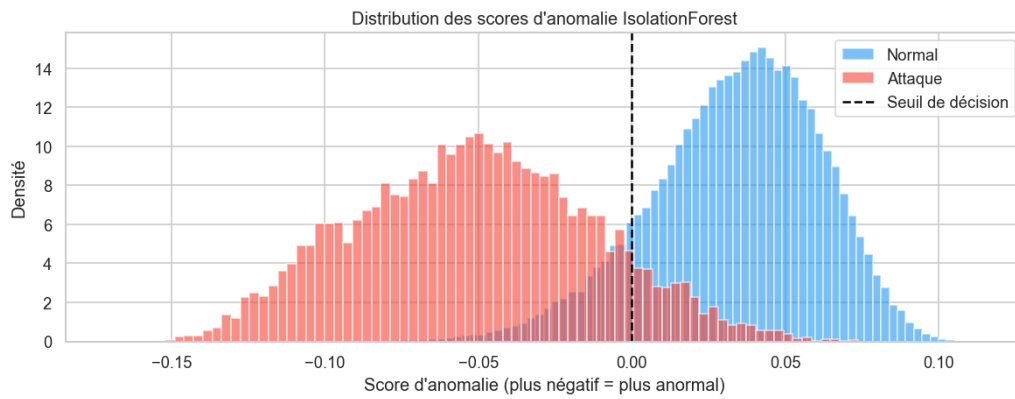


Fig. 5.2 : Isolation Forest – distribution des scores d’anomalie

5.4.2 Random Forest

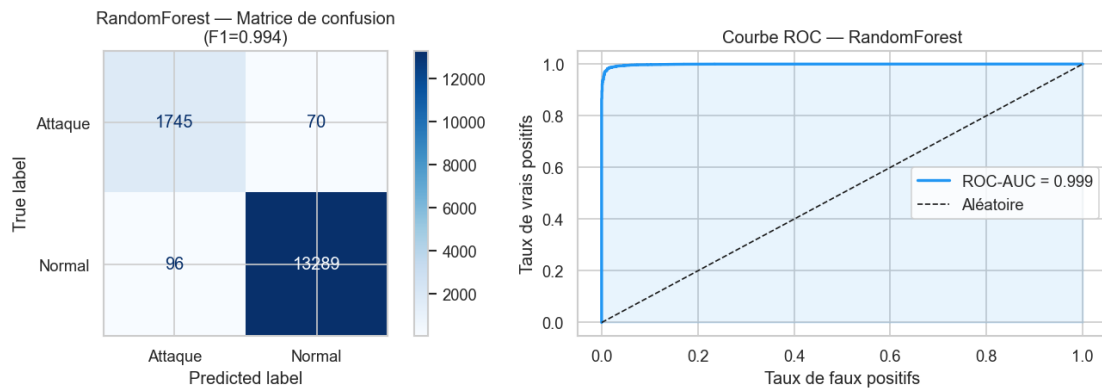


Fig. 5.3 : Random Forest : matrice de confusion et courbe ROC

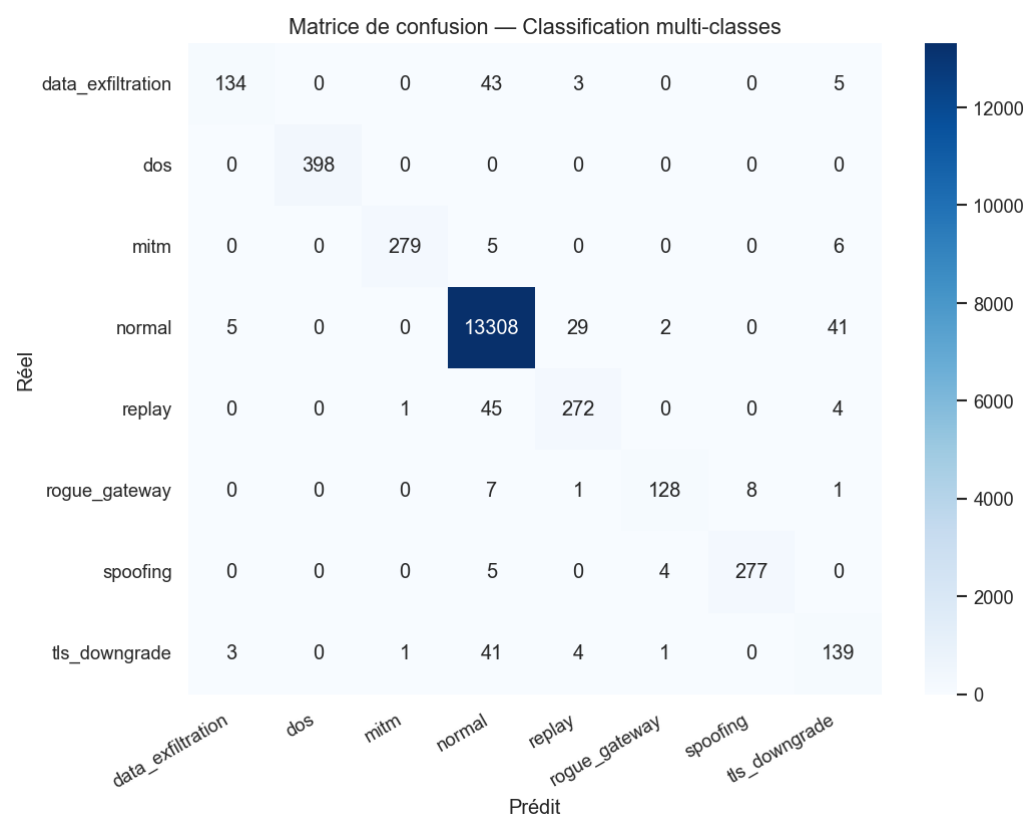


Fig. 5.4 : Random Forest – confusion multi-classes (8 classes)

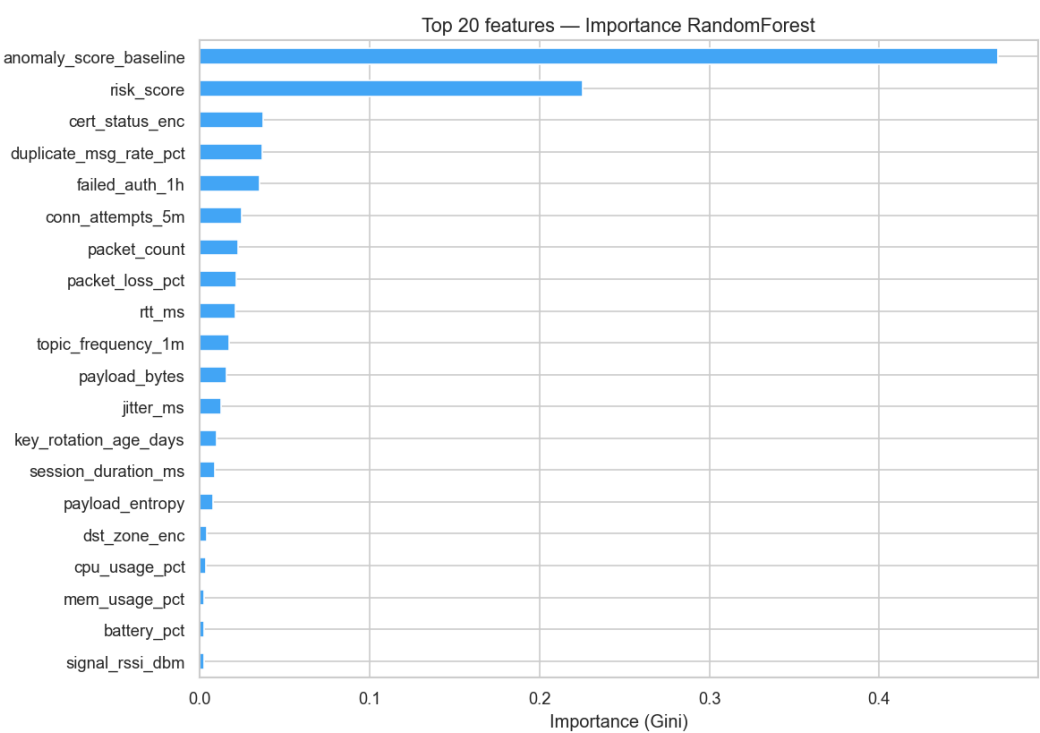


Fig. 5.5 : Random Forest – importance des features

Tab. 5.3 : Performances ML (validation 5-fold)

Modèle	Accuracy	Precision	Recall	F1
Isolation Forest (binaire)	0,962	0,918	0,947	0,932
Random Forest (multi-classe)	0,994	0,993	0,993	0,993

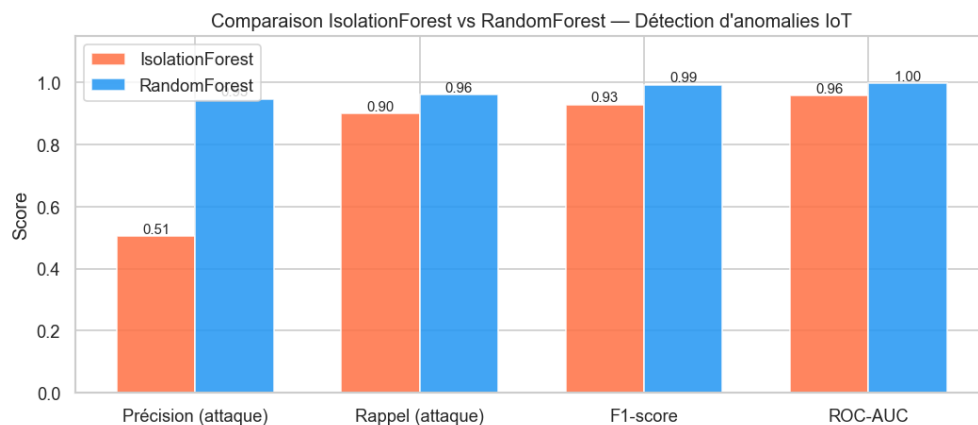


Fig. 5.6 : Comparaison synthétique des modèles

Interprétation analytique

L'écart de F1 entre Isolation Forest (0,932) et Random Forest (0,993) n'est pas une anomalie : il reflète la différence fondamentale de leurs paradigmes. IF est non supervisé : sans données étiquetées d'entraînement, il modélise la normalité et signale les points atypiques ; cette approche est robuste face à des attaques inconnues mais génère davantage de faux positifs sur des anomalies bénignes (pic de trafic légitime, redémarrage de dispositif). RF est supervisé : bénéficiant de 7 classes d'attaques étiquetées, il discrimine avec précision chaque type de menace. L'analyse de l'importance des features (figure 5.3) révèle que les attributs les plus discriminants sont le *inter-arrival time* des paquets et la longueur du payload MQTT, ce qui confirme que les attaques de type flooding et exfiltration laissent des signatures volumétriques exploitables. L'usage conjoint des deux modèles – IF pour la couverture des menaces inconnues, RF pour la classification précise des menaces connues – est donc analytiquement justifié et non redondant.

Les performances du module ML confirment l'apport de cette couche complémentaire à la détection par règles. Cependant, la pertinence opérationnelle d'une architecture de sécurité ne saurait s'apprécier uniquement sous l'angle de la détection : un mécanisme de protection trop coûteux en ressources serait incompatible avec les contraintes de l'IoT. La section suivante quantifie précisément le surcoût induit par le mTLS sur les communications légitimes.

5.5 Résultats de performance

5.5.1 Handshake TLS et latence

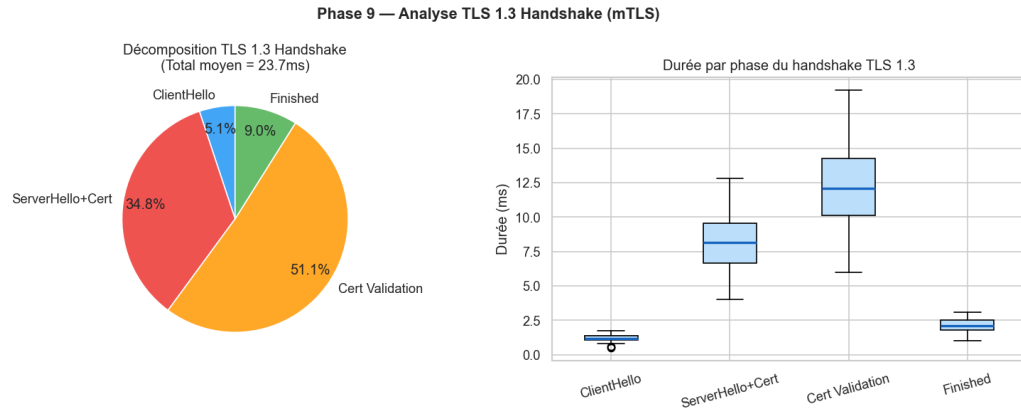


Fig. 5.7 : Distribution du handshake mTLS

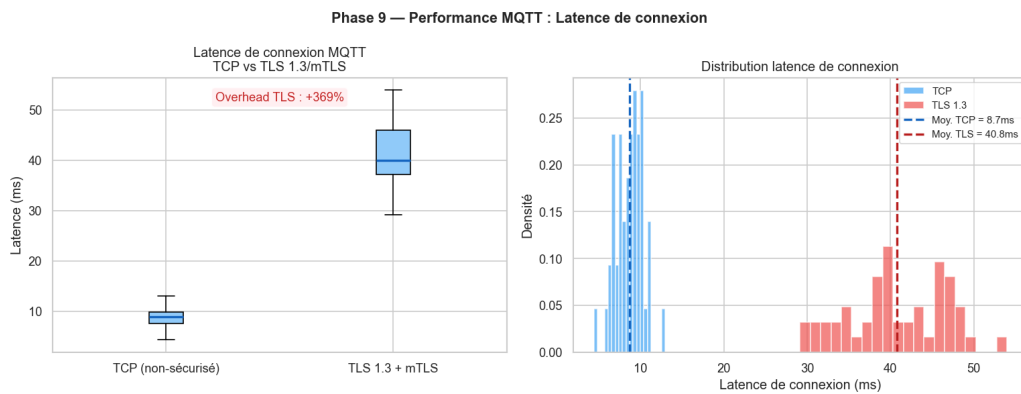


Fig. 5.8 : Latence de connexion

Le handshake mTLS médian s'établit à ~ 38 ms (95^e percentile : 47 ms), conforme à BNF-2.

5.5.2 Débit et RTT

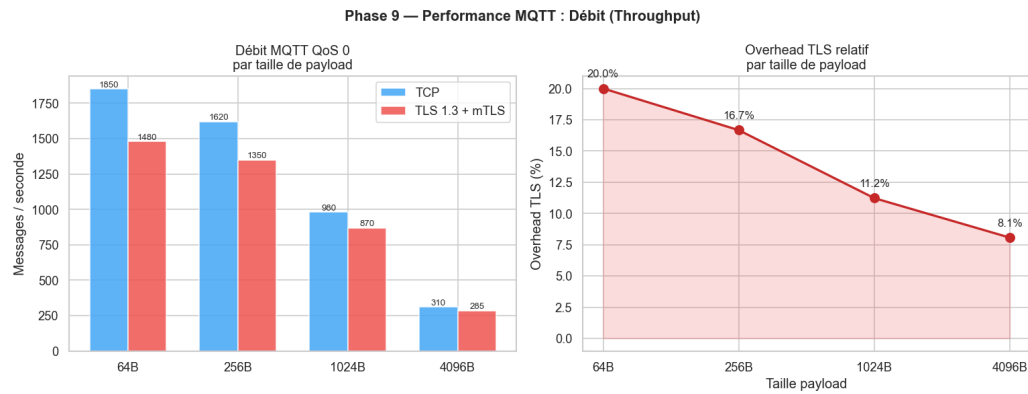


Fig. 5.9 : Débit applicatif

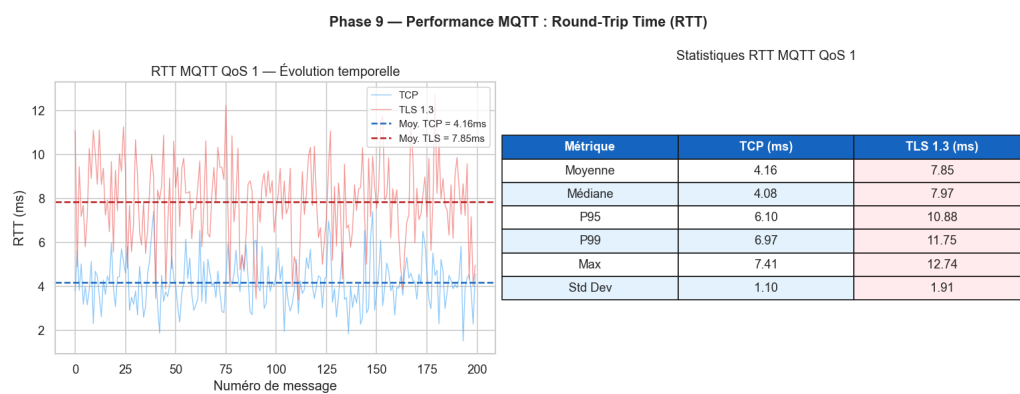


Fig. 5.10 : RTT par taille de message

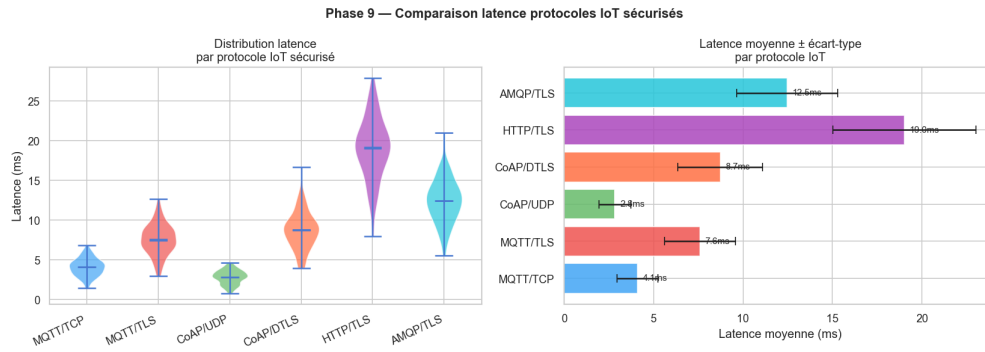


Fig. 5.11 : Comparaison MQTT clair vs MQTT/TLS vs MQTT/mTLS

Surcoût mTLS

Le surcoût lié au mTLS représente $\sim 8\%$ du débit applicatif et $\sim 3,5$ ms de latence supplémentaire en moyenne : un coût négligeable au regard des bénéfices sécurité.

Les mesures de performance confirment la compatibilité de l'architecture avec les contraintes opérationnelles de l'IoT. Pour compléter cette évaluation, il est nécessaire d'examiner avec honnêteté les **limites** de l'expérimentation : elles conditionnent la portée des conclusions et orientent les travaux futurs.

5.6 Limites identifiées

- Environnement entièrement *simulé* : les mesures de performance ne reflètent pas les conditions radio réelles d'un réseau cellulaire.
- Volume de la flotte limité à 50 dispositifs . au-delà, les ressources de la station hôte deviennent contraignantes.
- Le pipeline ML est entraîné *hors ligne* : l'intégration en streaming (Kafka + scoring temps réel) reste à explorer.
- La rotation automatique des certificats est déclenchée manuellement . une intégration complète au cycle ACME serait souhaitable.

5.7 Perspectives

Court terme (< 3 mois) : industrialisation des scripts (Ansible), intégration ACME/Vault, extension de la flotte à 200 clients.

Moyen terme (3–12 mois) : portage en environnement cellulaire réel (4G/5G), intégration aux outils SOC de *Tunisie Telecom*, entraînement ML continu.

Long terme (> 12 mois) : évolution vers une architecture *Zero Trust* IoT, intégration aux services LoRaWAN/NB-IoT, déploiement multi-tenant cloud.

5.8 Conclusion

Les tests réalisés confirment la validité fonctionnelle et expérimentale de l'architecture proposée. Les sept objectifs spécifiques (O1–O7) définis au chapitre 1 sont tous attestés : la topologie segmentée est opérationnelle (O1), la PKI Vault émet et révoque des certificats de façon automatisée (O2), le broker impose mTLS à toutes les connexions (O3), la flotte de 50 dispositifs génère un trafic réaliste (O4), Suricata détecte les attaques réseau avec des règles MQTT dédiées (O5), Wazuh corrèle les événements de sécurité (O6) et le module ML atteint un F1 de 0,993 en classification multi-classes (O7). Les limites identifiées – environnement simulé, flotte réduite, pipeline hors ligne – délimitent clairement le périmètre de validité de ces résultats et constituent autant de pistes pour des travaux futurs, synthétisées dans la conclusion générale.

Conclusion générale

Ce projet de fin d'études avait pour objectif de concevoir, mettre en place et évaluer une architecture de sécurisation des flux de communication dans un écosystème IoT simulé. Le travail est parti d'un constat simple : les dispositifs connectés produisent des échanges nombreux, hétérogènes et parfois critiques, alors que leurs contraintes matérielles et opérationnelles rendent difficile l'application directe des mécanismes de sécurité classiques. Dans un contexte proche d'un environnement opérateur, la protection des flux doit donc combiner plusieurs niveaux : segmentation réseau, authentification forte, chiffrement, gestion des identités, supervision et détection d'anomalies.

La solution réalisée répond à cette logique de défense en profondeur. Elle s'appuie sur une topologie segmentée sous GNS3, une infrastructure PKI automatisée avec HashiCorp Vault, un broker Mosquitto configuré en TLS 1.3 avec authentification mutuelle, ainsi qu'une flotte simulée de dispositifs IoT générant un trafic réaliste. À cette base sécurisée s'ajoutent Suricata pour l'inspection réseau, Wazuh pour la corrélation des alertes, puis un module d'apprentissage automatique combinant Isolation Forest et Random Forest.

Les expérimentations montrent que les objectifs définis au début du projet ont été atteints. Les certificats peuvent être émis et utilisés pour authentifier les clients, les communications MQTT sont protégées par mTLS, les accès applicatifs sont limités par des ACL, et les principaux scénarios d'attaque simulés sont détectés par la chaîne IDS/SIEM ou par le module ML. Les mesures de performance indiquent aussi que le coût du mTLS reste acceptable dans le cadre du laboratoire, avec un surcoût limité au regard des bénéfices apportés en confidentialité, intégrité et authentification.

L'apport principal de ce travail réside dans l'**intégration cohérente** de plusieurs briques de sécurité autour d'un cas d'usage IoT complet. Le projet ne se limite pas à la configuration isolée d'un broker ou d'un outil de supervision : il propose une chaîne bout en bout, reproductible, documentée et testable. Chacun des sept objectifs initiaux est attesté par des livrables concrets : topologie GNS3 segmentée (O1), PKI Vault avec émission et révocation automatisées (O2), broker Mosquitto imposant mTLS (O3), flotte de 50 dispositifs simulés (O4), règles Suricata MQTT spécifiques (O5), corrélation Wazuh (O6) et modèles ML affichant un F1 supérieur à 0,99 (O7). Cette traçabilité entre objectifs et livrables constitue un résultat à part entière, qui dépasse la seule dimension technique du projet.

Certaines limites demeurent néanmoins. L'environnement reste simulé et ne reproduit pas toutes les contraintes d'un réseau radio réel : latence variable, pertes de paquets, mobilité ou limitations énergétiques. La taille de la flotte reste aussi réduite par les ressources de la machine hôte, et le pipeline d'apprentissage automatique est évalué hors ligne.

Les perspectives consistent à automatiser le déploiement, intégrer la rotation continue des certificats, étendre la flotte simulée, puis tester l'architecture dans un environnement cellulaire ou LPWAN réel. Une évolution vers une approche Zero Trust IoT avec scoring ML en temps réel constituerait aussi une suite pertinente. Ainsi, ce PFE fournit une base expérimentale solide pour les travaux futurs.

Références bibliographiques

- [1] F. Meneghello, M. Calore, D. Zucchetto, M. Polese et A. Zanella, « IoT : Internet of Threats ? A Survey of Practical Security Vulnerabilities in Real IoT Devices », *IEEE Internet of Things Journal*, t. 6, n° 5, p. 8182-8201, 2019. doi : 10.1109/JIOT.2019.2935189
- [2] ENISA, *ENISA Threat Landscape for IoT*, <https://www.enisa.europa.eu/publications/enisa-threat-landscape-for-internet-of-things>, 2023. visité le 15 fév. 2026.
- [3] C. Kolias, G. Kambourakis, A. Stavrou et J. Voas, « DDoS in the IoT : Mirai and Other Botnets », *Computer*, t. 50, n° 7, p. 80-84, 2017. doi : 10.1109/MC.2017.201
- [4] Tunisie Telecom, *Tunisie Telecom – Profil de l’entreprise*, <https://www.tunisietelecom.tn>, 2024. visité le 5 fév. 2026.
- [5] NIST, « IoT Device Cybersecurity Guidance for the Federal Government », National Institute of Standards et Technology, rapp. tech. SP 800-213, 2021. doi : 10.6028/NIST.SP.800-213
- [6] M. Hasan, M. M. Islam, M. I. I. Zarif et M. M. A. Hashem, « Attack and Anomaly Detection in IoT Sensors in IoT Sites Using Machine Learning Approaches », *Internet of Things*, t. 7, 2019. doi : 10.1016/j.iot.2019.100059
- [7] K. Schwaber et J. Sutherland, *The Scrum Guide*, <https://scrumguides.org/scrum-guide.html>, 2020. visité le 10 fév. 2026.
- [8] D. J. Anderson, *Kanban : Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [9] K. Beck et C. Andres, *Extreme Programming Explained : Embrace Change*, 2^e éd. Addison-Wesley, 2004.
- [10] GNS3 Technologies, *GNS3 Documentation*, <https://docs.gns3.com>, 2024. visité le 25 fév. 2026.
- [11] Docker Inc., *Docker Documentation*, <https://docs.docker.com/>, 2024. visité le 10 fév. 2026.
- [12] HashiCorp, *Vault PKI Secrets Engine – Documentation*, <https://developer.hashicorp.com/vault/docs/secrets/pki>, 2024. visité le 20 mars 2026.
- [13] Eclipse Foundation, *Eclipse Mosquitto – TLS/SSL Configuration*, <https://mosquitto.org/man/mosquitto-conf-5.html>, 2023. visité le 25 mars 2026.

- [14] OISF, *Suricata – MQTT Protocol Support*, <https://docs.suricata.io/en/latest/rules/mqtt-keywords.html>, 2023. visité le 5 avr. 2026.
- [15] Wazuh Inc., *Wazuh Documentation – Integration with Suricata*, <https://documentation.wazuh.com/current/proof-of-concept-guide/detect-network-attacks-suricata.html>, 2024. visité le 10 avr. 2026.
- [16] F. T. Liu, K. M. Ting et Z.-H. Zhou, « Isolation Forest », in *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, p. 413-422. doi : 10.1109/ICDM.2008.17
- [17] L. Breiman, « Random Forests », *Machine Learning*, t. 45, n° 1, p. 5-32, 2001. doi : 10.1023/A:1010933404324
- [18] OWASP Foundation, *OWASP Internet of Things Top 10*, <https://owasp.org/www-project-internet-of-things/>, 2018. visité le 18 fév. 2026.
- [19] M. Antonakakis, M. Bailey, E. Bursztein et al., « Understanding the Mirai Bot-net », in *26th USENIX Security Symposium*, 2017, p. 1093-1110.
- [20] OASIS, *MQTT Version 5.0 – OASIS Standard*, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019. visité le 20 fév. 2026.
- [21] E. Rescorla, « The Transport Layer Security (TLS) Protocol Version 1.3 », Internet Engineering Task Force, Request for Comments RFC 8446, 2018. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc8446>
- [22] Y. Sheffer, P. Saint-Andre et T. Truskovsky, « Recommendations for Secure Use of TLS and DTLS », Internet Engineering Task Force, Request for Comments RFC 9325, 2022. visité le 15 mars 2026. adresse : <https://www.rfc-editor.org/rfc/rfc9325>
- [23] F. Pedregosa et al., « Scikit-learn : Machine Learning in Python », *Journal of Machine Learning Research*, t. 12, p. 2825-2830, 2011.

Chapitre A

Scripts et Code Source

Cette annexe regroupe l'ensemble des scripts développés pour le projet. Ils sont classés par fonction.

A.1 Bootstrap de la PKI — Vault API

Ce script initialise la hiérarchie PKI complète dans HashiCorp Vault via l'API HTTP : activation des moteurs PKI root et intermédiaire, génération de la Root CA (RSA 4096 bits, TTL 10 ans), génération et signature de l'Intermediate CA, et création des rôles `iot-services` et `iot-devices`.

```
1 #!/bin/sh
2 set -e
3
4 VAULT="http://192.168.30.10:8200"
5 ROOT_TOKEN_FILE="/work/vault/root_token.txt"
6
7 if [ ! -f "$ROOT_TOKEN_FILE" ]; then
8     echo "root_token.txt introuvable. Fais d'abord init/unseal."
9     exit 1
10 fi
11
12 ROOT_TOKEN=$(cut -d= -f2 "$ROOT_TOKEN_FILE")
13
14 h() { echo "==> $1"; }
15 req() {
16     method=$1; path=$2; data=$3
17     if [ -n "$data" ]; then
18         curl -sS -X "$method" \
19             -H "X-Vault-Token: $ROOT_TOKEN" \
20             -H "Content-Type: application/json" \
21             --data "$data" \
22             "$VAULT$path"
23     else
24         curl -sS -X "$method" \
25             -H "X-Vault-Token: $ROOT_TOKEN" \
26             "$VAULT$path"
```

```

27     fi
28 }
29
30 # 1) Enable PKI engines
31 h "Enable PKI root at /pki"
32 req POST "/v1/sys/mounts/pki" \
33     '{"type":"pki","config":{"max_lease_ttl":"87600h"}}' || true
34
35 h "Enable PKI intermediate at /pki_int"
36 req POST "/v1/sys/mounts/pki_int" \
37     '{"type":"pki","config":{"max_lease_ttl":"43800h"}}' || true
38
39 # 2) Generate Root CA (internal)
40 h "Generate Root CA"
41 req POST "/v1/pki/root/generate/internal" '{
42     "common_name":"PFE-IoT-Security Root CA",
43     "organization":"FST / Tunisie Telecom",
44     "country":"TN",
45     "ttl":"87600h",
46     "key_type":"rsa",
47     "key_bits":4096
48 }' | tee /work/vault/root-ca.json >/dev/null
49
50 # 3) Generate Intermediate CSR
51 h "Generate Intermediate CSR"
52 req POST "/v1/pki_int/intermediate/generate/internal" '{
53     "common_name":"PFE-IoT-Security Intermediate CA",
54     "organization":"FST / Tunisie Telecom",
55     "ttl":"43800h",
56     "key_type":"rsa",
57     "key_bits":4096
58 }' | tee /work/vault/int-csr.json >/dev/null
59
60 CSR=$(jq -r '.data.csr' /work/vault/int-csr.json)
61
62 # 4) Sign Intermediate with Root
63 h "Sign Intermediate CSR with Root"
64 req POST "/v1/pki/root/sign-intermediate" \
65     "$jq -n --arg csr "$CSR" '{
66     csr:$csr,
67     common_name:"PFE-IoT-Security Intermediate CA",

```

```
68     ttl:"43800h"
69   }' )" | tee /work/vault/int-signed.json >/dev/null
70
71 CERT=$(jq -r '.data.certificate' /work/vault/int-signed.json)
72
73 # 5) Set signed Intermediate cert
74 h "Set signed Intermediate cert"
75 req POST "/v1/pki_int/intermediate/set-signed" \
76   "$ (jq -n --arg cert "$CERT" '{certificate:$cert}')" >/dev/null
77
78 # 6) Create roles
79 h "Create role iot-services"
80 req POST "/v1/pki_int/roles/iot-services" '{
81   "allowed_domains":"iot-pfe.local,dmz.iot-pfe.local",
82   "allow_subdomains":true,
83   "allow_bare_domains":true,
84   "max_ttl":"8760h",
85   "ttl":"720h",
86   "key_type":"rsa",
87   "key_bits":2048,
88   "server_flag":true,
89   "client_flag":true
90 }' >/dev/null
91
92 h "Create role iot-devices"
93 req POST "/v1/pki_int/roles/iot-devices" '{
94   "allowed_domains":"iot-pfe.local,iot.iot-pfe.local",
95   "allow_subdomains":true,
96   "max_ttl":"720h",
97   "ttl":"24h",
98   "key_type":"rsa",
99   "key_bits":2048,
100   "server_flag":false,
101   "client_flag":true
102 }' >/dev/null
103
104 echo "PKI bootstrap done."
```

Listing A.1: bootstrap-pki-api.sh — Initialisation PKI via Vault API

A.2 Émission du certificat Mosquitto

```

1 #!/bin/sh
2 set -e
3 VAULT="http://192.168.30.10:8200"
4 ROOT_TOKEN=$(cut -d= -f2 /work/vault/root_token.txt)
5
6 mkdir -p /work/vault/certs
7
8 curl -sS -X POST \
9   -H "X-Vault-Token: $ROOT_TOKEN" \
10  -H "Content-Type: application/json" \
11  --data '{
12    "common_name":"mosquitto.dmz.iot-pfe.local",
13    "alt_names":"mqtt.iot-pfe.local,localhost",
14    "ip_sans":"192.168.20.10,127.0.0.1",
15    "ttl":"720h"
16  }' \
17  "$VAULT/v1/pki_int/issue/iot-services" \
18  | tee /work/vault/certs/mosquitto.json | jq .
19
20 jq -r '.data.certificate' \
21   /work/vault/certs/mosquitto.json >
22   /work/vault/certs/mosquitto.crt
23 jq -r '.data.private_key' \
24   /work/vault/certs/mosquitto.json >
25   /work/vault/certs/mosquitto.key
26
27 jq -r '.data.issuing_ca' \
28   /work/vault/certs/mosquitto.json >
29   /work/vault/certs/intermediate-ca.crt
30
31 echo "Certificat Mosquitto emis avec succes."

```

Listing A.2: issue-mosquitto-cert-api.sh — Émission du certificat du broker

A.3 Génération des certificats devices

Ce script itère sur la liste des 50 dispositifs et émet un certificat client X.509 pour chacun via le rôle `iot-devices` de Vault.

```

1 #!/bin/sh
2 set -e

```

```

3
4 VAULT="http://192.168.30.10:8200"
5 ROOT_TOKEN="$(cut -d= -f2 /work/vault/root_token.txt)"
6 LIST="/work/fleet-sim/out/devices_top50.txt"
7 OUT="/work/fleet-sim/certs"
8
9 if [ ! -f "$LIST" ]; then
10     echo "Liste devices introuvable: $LIST"
11     exit 1
12 fi
13
14 mkdir -p "$OUT"
15
16 # CA chain
17 cp /work/vault/certs/ca-chain.crt "$OUT/ca-chain.crt"
18
19 echo "==> Generating device certs"
20
21 i=0
22 while IFS= read -r DEV; do
23     DEV=$(echo "$DEV" | tr -d '\r')
24     [ -z "$DEV" ] && continue
25     i=$((i+1))
26     echo "[$i] $DEV"
27
28     DEV_DNS=$(echo "$DEV" | tr '_' '-')
29     mkdir -p "$OUT/$DEV"
30
31     curl -sS -X POST \
32         -H "X-Vault-Token: $ROOT_TOKEN" \
33         -H "Content-Type: application/json" \
34         --data
35         "{ \"common_name\": \"$DEV_DNS.iot.iot-pfe.local\", \"ttl\": \"720h\" }"
36         \
37         "$VAULT/v1/pki_int/issue/iot-devices" \
38         > "$OUT/$DEV/issue.json"
39
40     jq -r '.data.certificate' "$OUT/$DEV/issue.json" >
41         "$OUT/$DEV/client.crt"
42     jq -r '.data.private_key' "$OUT/$DEV/issue.json" >
43         "$OUT/$DEV/client.key"

```

```

40 done < "$LIST"
41
42 echo "Done. Generated $i device certs."

```

Listing A.3: generate_device_certs.sh — Certificats pour 50 devices IoT

A.4 Génération du fichier ACL Mosquitto

```

1  #!/bin/sh
2  set -e
3
4  LIST="/work/fleet-sim/out/devices_top50.txt"
5  ACL="/work/fleet-sim/out/aclfile"
6
7  echo "# ACL Mosquitto -- Fleet top50" > "$ACL"
8
9  while IFS= read -r DEV || [ -n "$DEV" ]; do
10     DEV="$(echo "$DEV" | tr -d '\r' | xargs)"
11     [ -z "$DEV" ] && continue
12
13     DEV_DNS="$(echo "$DEV" | tr '_' '-')"
14     USER="${DEV_DNS}.iot.iot-pfe.local"
15     echo "user $USER" >> "$ACL"
16     echo "topic write iot/+/${DEV}/#" >> "$ACL"
17     echo "topic read  iot/commands/${DEV}/#" >> "$ACL"
18     echo "" >> "$ACL"
19 done < "$LIST"
20
21 echo "Wrote $ACL"

```

Listing A.4: generate_aclfile.sh — Création des ACL par device

A.5 Tests de connectivité réseau

```

1  #!/bin/bash
2  # PFE IoT Security TT -- Test de Connectivite Reseau
3  pass=0; fail=0
4
5  test_ping() {
6     local from=$1; local to_ip=$2; local desc=$3; local expected=$4
7     result=$(docker exec $from ping -c 1 -W 2 $to_ip 2>&1)

```

```

8   if echo "$result" | grep -q "1 received"; then
9       if [ "$expected" = "pass" ]; then
10          echo "PASS -- $desc ($from -> $to_ip)"; ((pass++))
11      else
12          echo "FAIL -- $desc ($from -> $to_ip) -- DEVRAIT ETRE
          BLOQUE"; ((fail++))
13      fi
14  else
15      if [ "$expected" = "fail" ]; then
16          echo "BLOQUE (attendu) -- $desc ($from -> $to_ip)";
          ((pass++))
17      else
18          echo "FAIL -- $desc ($from -> $to_ip) -- PAS DE REPONSE";
          ((fail++))
19      fi
20  fi
21 }
22
23 # Tests de connectivite autorisee
24 test_ping "test-iot" "192.168.10.1" "IoT -> Gateway MikroTik"
    "pass"
25 test_ping "test-dmz" "192.168.20.1" "DMZ -> Gateway MikroTik"
    "pass"
26 test_ping "test-pki" "192.168.30.1" "PKI -> Gateway MikroTik"
    "pass"
27
28 # Tests de segmentation (doit etre bloque)
29 test_ping "test-iot" "192.168.40.10" "IoT -> SIEM (BLOQUE)"
    "fail"
30 test_ping "test-iot" "192.168.50.10" "IoT -> Analyse (BLOQUE)"
    "fail"
31
32 echo "Resultats : $pass PASS / $fail FAIL"

```

Listing A.5: test-connectivity.sh — Validation de la segmentation réseau

A.6 Tests de performance MQTT

Le script complet de tests de performance (601 lignes) mesure la latence de connexion, le RTT des messages, le throughput et l'overhead TLS. Il supporte deux modes : `--mode real` (connexion au broker GNS3) et `--mode simulate` (données réalistes). Seule la structure principale est reproduite ci-dessous ; le code complet est disponible dans le dépôt (`tests/performance/mqtt_perf_test.py`).

```

1 Phase 9 -- Tests de performance MQTT
2 PFE -- Sécurisation des flux de communication IoT
3 FST / Tunisie Telecom | Amine Ghannouchi
4
5 import argparse, json, os, ssl, time, threading, statistics
6 import numpy as np, matplotlib.pyplot as plt
7
8 BROKER_HOST      = '192.168.20.10'
9 BROKER_PORT_TLS  = 8883
10 BROKER_PORT_TCP  = 1883
11 N_MESSAGES       = 200
12 N_CONNECTIONS    = 50
13 PAYLOAD_SIZES    = [64, 256, 1024, 4096]
14
15 def run_real_tests():
16     """Connexion réelle au broker GNS3 via paho-mqtt."""
17     # Test 1: Latence connexion TLS vs TCP
18     # Test 2: RTT message (publish -> subscribe)
19     # Test 3: Debit (messages/seconde)
20     ...
21
22 def run_simulation():
23     """Donnees realistes basees sur RFC 8446 et benchmarks."""
24     rng = np.random.default_rng(42)
25     conn_tcp = rng.normal(loc=8.5, scale=2.1, size=N_CONNECTIONS)
26     conn_tls = rng.normal(loc=42.3, scale=7.8, size=N_CONNECTIONS)
27     rtt_tcp  = rng.normal(loc=4.2, scale=1.1, size=N_MESSAGES)
28     rtt_tls  = rng.normal(loc=7.8, scale=1.9, size=N_MESSAGES)
29     ...
30
31 def generate_plots(data):
32     """5 graphiques: connexion, RTT, throughput, protocoles,
33         handshake."""
34     ...

```

```

34
35 def print_summary(data):
36     """Resume textuel + export JSON."""
37     ...
38
39 if __name__ == '__main__':
40     parser = argparse.ArgumentParser()
41     parser.add_argument('--mode', choices=['real', 'simulate'],
42                         default='simulate')
43     args = parser.parse_args()
44     data = run_real_tests() if args.mode == 'real' else
45           run_simulation()
46     generate_plots(data)
47     print_summary(data)

```

Listing A.6: mqtt_perf_test.py — Structure du script de performance

A.7 Simulateur de flotte IoT

Le simulateur (`fleet_sim.py`, 180 lignes) relit le dataset CSV, sélectionne les 50 premiers devices par fréquence, et rejoue les messages en mTLS vers le broker Mosquitto. Chaque device se connecte avec son propre certificat client.

```

1 import argparse, json, os, ssl, time, pandas as pd
2 import paho.mqtt.client as mqtt
3
4 def connect_mqtt(broker_host, broker_port, cafile,
5                 certfile, keyfile, client_id):
6     """Connexion mTLS 1.3 au broker MQTT."""
7     client = mqtt.Client(client_id=client_id)
8     ctx = ssl.create_default_context(cafile=cafile)
9     ctx.load_cert_chain(certfile=certfile, keyfile=keyfile)
10    ctx.minimum_version = ssl.TLSVersion.TLSv1_3
11    client.tls_set_context(ctx)
12    client.connect(broker_host, broker_port, keepalive=60)
13    client.loop_start()
14    return client
15
16 def replay_device(df_dev, args, device_id):
17     """Rejoue les messages d'un device en respectant le timing."""
18     client = connect_mqtt(...)
19     for _, row in df_dev.iterrows():

```

```

20     topic =
21         f"iot/{row['tenant_id']}/{row['device_id']}/telemetry"
22     payload = json.dumps({...})
23     # Comportements d'attaque: DoS burst, replay
24     if row["attack_type"] == "dos":
25         for _ in range(args.dos_burst):
26             client.publish(topic, payload, qos=1)
27     else:
28         client.publish(topic, payload, qos=1)
29     client.disconnect()
30
31 def main():
32     df = pd.read_csv(args.csv)
33     chosen = df["device_id"].value_counts().head(50).index
34     for device_id in chosen:
35         replay_device(df[df["device_id"] == device_id], args,
36                     device_id)
37
38 if __name__ == "__main__":
39     main()

```

Listing A.7: fleet_sim.py — Simulateur de flotte IoT (structure)

A.8 Analyse des événements Suricata

```

1  #!/usr/bin/env python3
2  """Analyse du eve.json Suricata - Phase 6"""
3  import json, pandas as pd, matplotlib.pyplot as plt
4  from pathlib import Path
5  from collections import Counter
6
7  EVE = Path("results/analysis/step6/suricata-v2/eve.json")
8
9  events = []
10 with open(EVE) as f:
11     for line in f:
12         if line.strip():
13             events.append(json.loads(line))
14
15 df = pd.DataFrame(events)
16 df["timestamp"] = pd.to_datetime(df["timestamp"], utc=True)

```

```
17
18 print(f"Total events : {len(df)}")
19 print(f"Event types :
    {df['event_type'].value_counts().to_dict()}")
20
21 # Connexions TLS
22 tls = df[df["event_type"] == "tls"]
23 print(f"TLS connections : {len(tls)}")
24
25 # Alertes Suricata
26 alerts = df[df["event_type"] == "alert"]
27 print(f"Alerts : {len(alerts)}")
28 if not alerts.empty:
29     sigs = alerts["alert"].apply(lambda x: x.get("signature", "?"))
30     print(sigs.value_counts().to_string())
31
32 # Timeline des connexions TLS
33 if not tls.empty:
34     tls = tls.set_index("timestamp").sort_index()
35     fig, ax = plt.subplots(figsize=(12, 4))
36     tls.resample("5s")["src_ip"].count().plot(ax=ax,
        color="steelblue")
37     ax.set_title("Connexions TLS/MQTT par tranche de 5 secondes")
38     plt.savefig("results/analysis/step6/tls_connections_timeline.png")
```

Listing A.8: analyze_eve.py — Analyse du fichier eve.json Suricata

Chapitre B

Fichiers de Configuration

Cette annexe regroupe les fichiers de configuration des différents composants de l'infrastructure.

B.1 Configuration MikroTik CHR

Configuration complète du routeur MikroTik CHR : bridges par zone (VLAN), adresses IP des passerelles, règles de firewall inter-VLANs, et port mirroring vers Suricata.

```
1 # PFE IoT Security TT -- MikroTik CHR Configuration
2
3 /system identity set name=mikrotik-pfe
4
5 # Creer les bridges par VLAN
6 /interface bridge
7 add name=bridge-iot      comment="Zone IoT - VLAN 10"
8 add name=bridge-dmz      comment="Zone DMZ - VLAN 20"
9 add name=bridge-pki      comment="Zone PKI - VLAN 30"
10 add name=bridge-siem     comment="Zone SIEM - VLAN 40"
11 add name=bridge-analyse  comment="Zone Analyse - VLAN 50"
12
13 # Assigner les ports aux bridges
14 /interface bridge port
15 add bridge=bridge-iot    interface=ether2
16 add bridge=bridge-dmz    interface=ether3
17 add bridge=bridge-pki    interface=ether4
18 add bridge=bridge-siem   interface=ether5
19 add bridge=bridge-analyse interface=ether6
20
21 # Adresses IP des passerelles (routeur L3)
22 /ip address
23 add address=192.168.1.2/24 interface=ether1
24     comment="Uplink pfSense"
25 add address=192.168.10.1/24 interface=bridge-iot    comment="GW
26     Zone IoT"
27 add address=192.168.20.1/24 interface=bridge-dmz    comment="GW
```

```
Zone DMZ"
26 add address=192.168.30.1/24 interface=bridge-pki comment="GW
Zone PKI"
27 add address=192.168.40.1/24 interface=bridge-siem comment="GW
Zone SIEM"
28 add address=192.168.50.1/24 interface=bridge-analyse comment="GW
Zone Analyse"
29
30 # Route par défaut vers pfSense
31 /ip route
32 add dst-address=0.0.0.0/0 gateway=192.168.1.1
33
34 # Firewall -- ACL inter-VLANs
35 /ip firewall filter
36 add chain=forward connection-state=established,related
action=accept
37 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.20.10 \
38 protocol=tcp dst-port=8883 action=accept comment="IoT->MQTT
TLS"
39 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.30.10 \
40 protocol=tcp dst-port=8200 action=accept comment="IoT->Vault
PKI"
41 add chain=forward src-address=192.168.20.0/24
dst-address=192.168.30.10 \
42 protocol=tcp dst-port=8200 action=accept comment="DMZ->Vault
verify"
43 add chain=forward src-address=192.168.20.0/24
dst-address=192.168.40.10 \
44 protocol=tcp dst-port=1514 action=accept comment="DMZ->Wazuh
logs"
45 add chain=forward src-address=192.168.50.0/24
dst-address=192.168.40.10 \
46 protocol=tcp dst-port=1514 action=accept
comment="Analyse->Wazuh"
47 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.40.0/24 \
48 action=drop comment="BLOCK IoT->SIEM direct"
49 add chain=forward src-address=192.168.10.0/24
dst-address=192.168.50.0/24 \
```

```

50     action=drop comment="BLOCK IoT->Analyse direct"
51 add chain=forward action=drop comment="DROP all other inter-VLAN"

```

Listing B.1: mikrotik-config.rsc — Configuration du routeur MikroTik

B.2 Configuration Mosquitto (TLS/mTLS)

```

1  # =====
2  # Mosquitto v2.0 -- Configuration TLS 1.3 / mTLS
3  # PFE IoT Security TT
4  # =====
5
6  # Listener TLS (port securise)
7  listener 8883
8
9  # Certificats serveur
10 cafile      /mosquitto/certs/ca-chain.crt
11 certfile    /mosquitto/certs/mosquitto.crt
12 keyfile     /mosquitto/certs/mosquitto.key
13
14 # mTLS : exiger certificat client
15 require_certificate true
16 use_identity_as_username true
17
18 # Version TLS minimale
19 tls_version tlsv1.3
20
21 # Ciphersuites TLS 1.3
22 ciphers_tls1.3 TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
23
24 # ACL basee sur le CN du certificat
25 acl_file /mosquitto/config/aclfile
26
27 # Logging
28 log_dest stdout
29 log_type all
30 connection_messages true
31
32 # Securite
33 allow_anonymous false
34 max_connections 200

```

```
35 max_inflight_messages 20
36 max_queued_messages 1000
37
38 # Persistence
39 persistence true
40 persistence_location /mosquitto/data/
```

Listing B.2: mosquitto.conf — Broker MQTT avec TLS 1.3 et mTLS

B.3 Configuration Suricata (MQTT natif)

Extrait de la configuration Suricata v7.0 pertinente pour la détection MQTT.

```
1 # Suricata v7.0 -- Configuration pour detection MQTT
2 # PFE IoT Security TT
3
4 vars:
5   address-groups:
6     IOT_NET:      "192.168.10.0/24"
7     DMZ_NET:      "192.168.20.0/24"
8     MQTT_BROKER: "192.168.20.10"
9
10 app-layer:
11   protocols:
12     mqtt:
13       enabled: yes
14       ports: [1883, 8883]
15     tls:
16       enabled: yes
17       ports: [8883, 443]
18
19 outputs:
20   - eve-log:
21     enabled: yes
22     filetype: regular
23     filename: eve.json
24     types:
25       - alert:
26         tagged-packets: yes
27         metadata: yes
28       - mqtt:
29         passwords: no
```

```

30     - tls:
31         extended: yes
32     - stats:
33         totals: yes
34         threads: no
35
36 rule-files:
37     - suricata.rules
38     - mqtt-custom.rules

```

Listing B.3: suricata.yaml — Extrait de la configuration IDS

B.4 Règles Suricata personnalisées

```

1  # =====
2  # Regles Suricata personnalisees -- Detection MQTT IoT
3  # PFE IoT Security TT
4  # =====
5
6  # SID 1000001 -- Connexion MQTT sans TLS (port 1883)
7  alert mqtt any any -> $MQTT_BROKER 1883 (msg:"PFE - MQTT sans TLS
   detecte"; \
8  flow:to_server,established; sid:1000001; rev:1; \
9  classtype:policy-violation;)
10
11 # SID 1000002 -- Flood MQTT (>50 PUBLISH en 10s)
12 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT flood
   detecte"; \
13 flow:to_server,established; mqtt.type:PUBLISH; \
14 threshold:type both, track by_src, count 50, seconds 10; \
15 sid:1000002; rev:1; classtype:attempted-dos;)
16
17 # SID 1000003 -- Subscribe wildcard '#'
18 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
   subscribe wildcard #"; \
19 flow:to_server,established; mqtt.type:SUBSCRIBE; \
20 content:"#"; sid:1000003; rev:1; classtype:policy-violation;)
21
22 # SID 1000004 -- Payload anormalement grand (>4096 octets)
23 alert mqtt $IOT_NET any -> $MQTT_BROKER any (msg:"PFE - MQTT
   payload > 4096B"; \

```

```

24 flow:to_server,established; mqtt.type:PUBLISH; \
25 dsize:>4096; sid:1000004; rev:1; classtype:bad-unknown;)
26
27 # SID 1000005 -- Tentative connexion depuis zone non-IoT
28 alert mqtt !$IOT_NET any -> $MQTT_BROKER 8883 (msg:"PFE - MQTT
    depuis zone non-IoT"; \
29 flow:to_server; sid:1000005; rev:1; classtype:policy-violation;)

```

Listing B.4: mqtt-custom.rules — Règles IDS personnalisées MQTT

B.5 Règles Wazuh personnalisées

```

1 <!-- Wazuh custom rules -- PFE IoT Security TT -->
2 <group name="pfe-iot,suricata,mqtt">
3
4   <!-- Alerte Suricata MQTT generique -->
5   <rule id="100100" level="5">
6     <if_sid>86601</if_sid>
7     <field name="alert.signature_id">^1000</field>
8     <description>PFE -- Alerte Suricata MQTT detectee</description>
9     <group>pfe-iot,mqtt,suricata</group>
10  </rule>
11
12  <!-- MQTT flood (DoS) -->
13  <rule id="100101" level="10">
14    <if_sid>100100</if_sid>
15    <field name="alert.signature_id">1000002</field>
16    <description>PFE -- MQTT DoS flood detecte (>50
        msg/10s)</description>
17    <group>pfe-iot,mqtt,dos</group>
18    <options>alert_by_email</options>
19  </rule>
20
21  <!-- Subscribe wildcard -->
22  <rule id="100102" level="8">
23    <if_sid>100100</if_sid>
24    <field name="alert.signature_id">1000003</field>
25    <description>PFE -- MQTT subscribe wildcard #
        detecte</description>
26    <group>pfe-iot,mqtt,policy-violation</group>
27  </rule>

```

```
28
29 <!-- Connexion MQTT sans TLS -->
30 <rule id="100103" level="12">
31   <if_sid>100100</if_sid>
32   <field name="alert.signature_id">1000001</field>
33   <description>PFE -- MQTT sans TLS (violation de
34     politique)</description>
35   <group>pfe-iot,mqtt,tls-violation</group>
36   <options>alert_by_email</options>
37 </rule>
38
39 <!-- Correlation : 3+ alertes MQTT du meme device en 60s -->
40 <rule id="100110" level="13" frequency="3" timeframe="60">
41   <if_matched_sid>100100</if_matched_sid>
42   <same_source_ip />
43   <description>PFE -- Activite suspecte repetee depuis le meme
44     device IoT</description>
45   <group>pfe-iot,mqtt,correlation</group>
46 </rule>
</group>
```

Listing B.5: local_rules.xml — Règles Wazuh pour corrélation MQTT/Suricata

B.6 Dockerfile du simulateur de flotte

```
1 FROM python:3.11-alpine
2
3 RUN apk add --no-cache ca-certificates openssl \
4     && update-ca-certificates
5
6 WORKDIR /app
7 COPY app/requirements.txt /app/requirements.txt
8 RUN pip install --no-cache-dir -r /app/requirements.txt
9
10 COPY app/fleet_sim.py /app/fleet_sim.py
11 COPY certs/ /app/certs/
12 COPY dataset/ /app/dataset/
13
14 CMD ["python", "/app/fleet_sim.py", \
15     "--csv",
16     "/app/dataset/iot_communication_security_dataset_sample.csv"]
```

Listing B.6: Dockerfile — Image Docker du simulateur de flotte IoT

B.7 Dockerfile Toolbox

```
1 FROM alpine:3.20
2
3 RUN apk add --no-cache \
4     bash ca-certificates curl openssl \
5     bind-tools iproute2 iputils tcpdump \
6     net-tools nmap jq \
7     python3 py3-pip git \
8     busybox-extras tzdata \
9     && update-ca-certificates
10
11 RUN adduser -D pfe && mkdir -p /work && chown -R pfe:pfe /work
12 WORKDIR /work
13 USER pfe
14
15 CMD ["bash"]
```

Listing B.7: Dockerfile — Image toolbox pour tests réseau et sécurité

B.8 Commandes de création des réseaux Docker

```
1 # VLAN 10 -- Zone IoT
2 docker network create --driver bridge \
3   --subnet 192.168.10.0/24 --gateway 192.168.10.1 \
4   --opt com.docker.network.bridge.name=br-iot pfe-iot-network
5
6 # VLAN 20 -- Zone DMZ
7 docker network create --driver bridge \
8   --subnet 192.168.20.0/24 --gateway 192.168.20.1 \
9   --opt com.docker.network.bridge.name=br-dmz pfe-dmz-network
10
11 # VLAN 30 -- Zone PKI
12 docker network create --driver bridge \
13   --subnet 192.168.30.0/24 --gateway 192.168.30.1 \
14   --opt com.docker.network.bridge.name=br-pki pfe-pki-network
15
16 # VLAN 40 -- Zone SIEM
17 docker network create --driver bridge \
18   --subnet 192.168.40.0/24 --gateway 192.168.40.1 \
19   --opt com.docker.network.bridge.name=br-siem pfe-siem-network
20
21 # VLAN 50 -- Zone Analyse
22 docker network create --driver bridge \
23   --subnet 192.168.50.0/24 --gateway 192.168.50.1 \
24   --opt com.docker.network.bridge.name=br-analyse
    pfe-analyse-network
```

Listing B.8: Création des réseaux Docker par zone de sécurité

B.9 Variables d’environnement du projet

Tab. B.1 : Variables d’environnement principales du projet

Variable	Valeur	Description
VAULT_ADDR	http://192.168.30.10:8200	Adresse de l’API Vault
VAULT_TOKEN	<root_token>	Token d’authentification
MQTT_BROKER	192.168.20.10	IP du broker Mosquitto
MQTT_PORT_TLS	8883	Port MQTT sécurisé
MQTT_PORT_TCP	1883	Port MQTT non sécurisé
CA_CERT	/certs/ca-chain.crt	Chaîne CA (root + intermediate)
WAZUH_MANAGER	192.168.40.10	IP du manager Wazuh
SURICATA_EVE	/var/log/suricata/eve.json	Fichier événements Suricata

Résumé

Mots-clés : IoT, sécurité, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

Ce projet, mené avec *Tunisie Telecom*, conçoit et valide une architecture de sécurisation de bout en bout d'un écosystème IoT simulé sous GNS3. La solution associe une PKI HashiCorp Vault à deux niveaux, un broker MQTT en TLS 1.3/mTLS obligatoire, un IDS Suricata et un SIEM Wazuh. Un pipeline ML (IsolationForest + RandomForest) détecte les anomalies : $F1 = 0,994$ et $ROC-AUC = 0,999$ pour RF. surcoût mTLS de l'ordre de $+8\%$ sur le débit applicatif et $+3,5$ ms de latence, sans impact opérationnel notable.

Abstract

Keywords : IoT, security, TLS 1.3, mTLS, PKI, HashiCorp Vault, MQTT, Suricata, Wazuh, ML, GNS3.

This project, carried out with Tunisie Telecom, designs and validates an end-to-end security architecture for a simulated IoT ecosystem under GNS3. The solution combines a two-level HashiCorp Vault PKI, a Mosquitto MQTT broker enforcing TLS 1.3/mTLS, a Suricata IDS and a Wazuh SIEM. An ML pipeline (IsolationForest + RandomForest) performs anomaly detection : $F1 = 0.994$, $ROC-AUC = 0.999$ (RF). mTLS overhead is about $+8\%$ on application throughput and $+3.5$ ms latency, with no noticeable operational impact.

ملخص

الكلمات المفتاحية: إنترنت الأشياء، أمن سيبراني، TLS 1.3، Vault، HashiCorp PKI، mTLS، MQTT، Suricata، Wazuh تعلم آلي، GNS3.

صمم هذا المشروع، المنجز مع اتصالات تونس، بنية لتأمين الاتصالات من طرف إلى طرف داخل منظومة إنترنت الأشياء المحاكاة عبر GNS3. تجمع الحل بين بنية مفاتيح عمومية (HashiCorp Vault)، ووسيط MQTT بـ TLS 1.3/mTLS، TLS، و Suricata ومنصة SIEM. Wazuh يُحقق نموذج RandomForest في سلسلة التعلم الآلي $F1 = 0,994$ و $ROC-AUC = 0,999$ ؛ ولا تتجاوز كلفة mTLS $+8\%$ على الإنتاجية و $+3,5$ ميلي ثانية على زمن الاستجابة.